SUP'COM

المدرسة العليا للمواصلات بتونس

ÉCOLE SUPÉRIEURE DES COMMUNICATIONS DE TUNIS

Telecommunications engineering Cycle

Graduation Project Report

Topic :

« Fixed complexity tiny reservoir heterogeneous network for on-line ECG learning of anomalies»

Realized by:

Norhen ABDENNADHER

Supervisors :

Mr. Danilo Pau
Mr. Gian Antonio Susto
Mr. Mourad Zghal

Co-Supervisor :

Mr. Arcangelo Bruna

Work proposed and realized in collaboration with:

life.augmented

UNIVERSITÀ DEGLI STUDI DI PADOVA

Année universitaire : 2020-2021

Abstract

This report summarizes the work carried out as part of the end of studies project to obtain the Tunisian national engineering degree in telecommunications, performed as an internship in collaboration with STMicroelectronics Milan, Italy and the University of Padova.

The electrocardiogram (ECG), being one of the most extensively used signals to monitor cardiovascular diseases, captures the heart's arrhythmia. Patients with such pathology are often monitored for extended periods of time, requiring data storage, and a very time-consuming offline search of anomalies. In this report, we propose an automated deep learning pipeline based on reservoir computing, followed by principal component analysis and one-class support vector machine, that can be used to perform on-line learning and real-time anomaly detection of pathological conditions and therefore to raise warnings. The learning step requires fixed computational complexity and memory that are affordable on a low-power microcontroller unit (MCU). During the learning phase, it uses a very limited amount of input data, e.g., 10 KB, which makes this work suitable for fast customization when device restart is required. The detection accuracy has been evaluated on the publicly available MIT-BIH arrhythmia dataset, also modified to mimic the temporal sliding of input tensors on the ECG streaming data.

Keywords: ECG, Time series data, Reservoir computing, Recurrent network, Anomaly detection, Online learning, Principal component analysis, One class support vector machine, Sliding window...

Résumé

Ce rapport résume le travail mené dans le cadre du projet de fin d'études de l'Ecole Supérieure des Communications de Tunis (SUPCOM) pour l'obtention du diplôme national d'ingénieur en télécommunications, effectué pendant un stage en collaboration avec STMicroelectronics Milan, Italie et l'Université de Padoue.

L'électrocardiogramme (ECG), est l'un des signaux les plus utilisés pour surveiller les maladies cardiovasculaires, et diagnostiquer un problème cardiaque. Les patients atteints d'une telle pathologie sont souvent surveillés pendant une longue période de temps, ce qui nécessite un stockage de données et une recherche hors ligne des anomalies très longue. Dans ce rapport, nous proposons un pipeline d'apprentissage en profondeur automatisé basé sur un Reservoir Computing, suivi d'un module de réduction de dimension et de one-class support vector machine. Ce modéle peut être utilisé pour effectuer un apprentissage en ligne et une détection d'anomalies en temps réel des pathologies cardiaques. L'étape d'apprentissage nécessite une complexité de calcul fixe et une mémoire abordable sur un microcontrôleur à faible consommation. Pendant la phase d'apprentissage, on utilise une quantité très limitée de données, par exemple 10 Ko, ce qui rend ce travail approprié pour une personnalisation rapide lorsque le redémarrage de l'appareil est requis.

Mots-clés: Électrocardiogramme, données de séries chronologiques, Reservoir computing, Réseau récurrent, Détection d'anomalie, Apprentissage en ligne, Principal component analysis, One class support vector machine, Fenêtre glissante...

Dedications

“

To my family, thank you for all your supports, help, prayers, and advice, thank you for believing in me, for being strong and patient, for appreciating my achievements, and finally, thank you for what you taught me to be the person I am today. This work is the culmination of your great efforts. May God Bless You with Good Health and Long Life,

To my parents, Tarek and Sihem, so many words cannot express my love and affection. You have always supported and encouraged me throughout my journey. On this memorable day, for me as well as for you, receive this work as a sign of my deep appreciation and my profound esteem,

To my brother and sister, Riadh and Rihab, I can never imagine how the world would be without you. Both of you have been my best cheerleaders. Wishing you all luck and success in your lives,

To someone close to my heart and far from the eyes, someone who was there for me in my ups and downs, in my joy and sadness. To my beloved Neji, the one I always rely on when I felt anxious and stressed, I cannot express my deepest gratitude for the continuous support you have always shown, for always believing in my skills and my competence. Thank you for everything you have added to my life.

To all my dear ones, to all of you

Thank you.

”

- Norhen

Acknowledgement

I have been lucky enough to have a chance to spend a few months working at STMicroelectronics Milan, Italy. A rich and highly dynamic learning environment provided me with valuable hands-on experience and a solid knowledge base for my current and future work.

I would like to take this opportunity to express my heartfelt gratitude to a number of people whose I was blessed to know and work with. My 18 years of studies had come to a successful conclusion thanks to you.

First, to my supervisors and coordinators **Prof. Mourad Zghal** and **Prof. Marco Santagiustina** for making this work possible today and for their valuable support and advice during all the periods of mobility.

To my thesis Tutor at the University of Padova **Prof Gian Antonio Susto**, for his advice and help.

To my thesis Tutor at STMicroelectronics company **Mr. Danilo Pau**, the one I turn to whenever I run into a trouble spot, he was an incredible mentor to me. I can't forget his generosity and his kind support, especially under very difficult conditions. He was always there for me to provide me with his valuable suggestions and wise words to make my internship succeed and faces all the encountered problems. Thank you for all the lessons you've given me, for teaching me the best way to work smarter, for trusting my skills, and for handing me the keys to the professional world. Without your advice and encouragement, I would not have made it this far.

To my thesis co-tutor **Mr.Arcangelo Bruna** thank you for sharing knowledge and for making me full of hope and energy with your motivational feedback. I would like to thank you for being there when I needed help.

Thanks to all of the SUPCOM and UNIPD teachers who were involved in my education during my time at university.

I would like to express my gratitude to the members of the jury for the honor they have bestowed upon me by reviewing my graduation project, and I hope that they can find the clarification and inspiration that they seek in this report.

Contents

Abstract . . . . . **I**

Résumé . . . . . **II**

Dedications . . . . . **III**

Acknowledgement . . . . . **IV**

General Introduction **1**

1 Internship context & Scope Definition **3**
1.1 The host enterprise 4
1.1.1 Enterprise presentation 4
1.1.2 Sector of activity 5
1.1.3 STMicroelectronics, Milan Italy 5
1.2 Thesis overview and problem statement 6
1.2.1 Problem Statement & Motivation 6
1.2.2 Objectives & Proposed solution 6
1.3 Project management approach 7
1.3.1 Meetings & team work 8
1.3.2 AI projects management methodology description 8
1.4 Conclusion 9

2 Background Theory **10**
2.1 ECG signal 11
2.2 Arrhythmia 12
2.2.1 What is Arrhythmia 12
2.2.2 Types of arrhythmia 12
2.3 Time series data 13
2.3.1 Definition 13
2.3.2 Time series examples 13
2.4 Anomaly Detection 14
2.4.1 Definition 14
2.4.2 Anomaly detection with ML 15
2.4.2.1 Supervised learning of anomalies 15
2.4.2.2 Semi-supervised Anomaly Detection 15
2.4.2.3 Unsupervised learning of anomalies 16
2.5 On-line learning 16

2.6 Artificial Neural Network . . . 17
2.6.1 Definition of an ANN . . . 17
2.6.2 How do Neural Networks work . . . 18
2.7 Recurrent Neural Network . . . 20
2.8 Conclusion . . . 21

3 Datasets understanding and preparation process . . . 22
3.1 Datasets . . . 23
3.1.1 MIT BIH arrhythmia database . . . 23
3.1.1.1 ECG lead configuration . . . 24
3.1.1.2 MIT BIH annotation process . . . 24
3.1.2 STM custom dataset . . . 26
3.2 Data preparation and modification . . . 28
3.2.1 Intuition and requirements . . . 28
3.2.2 Sliding window . . . 28
3.2.3 Modified dataset with sliding windows . . . 28
3.2.3.1 Derived MIT BIH database . . . 29
3.2.3.2 Derived ST custom dataset . . . 29
3.2.3.3 Reannotation process . . . 30
3.2.4 Programming tools . . . 31
3.3 Conclusion . . . 32

4 Proposed network architecture . . . 33
4.1 HyRCoML topology . . . 34
4.1.1 Reservoir module . . . 35
4.1.1.1 Sparcity . . . 36
4.1.1.2 Bidirectionality . . . 37
4.1.2 Representation module . . . 38
4.1.3 Dimensionality reduction module . . . 38
4.1.4 Readout OC-SVM module . . . 40
4.1.4.1 OC-SVM for Imbalanced Data . . . 40
4.1.4.2 Basic concepts of One Class Support Vector Machines . . . 41
4.1.4.3 Kernel function . . . 42
4.2 LSTM based architecture . . . 43
4.2.1 LSTM overview . . . 44
4.2.2 LSTM autoencoder benchmark architecture . . . 44
4.3 Implementation Details . . . 46
4.3.1 Hardware Environment . . . 46
4.3.2 Software Environment . . . 48
4.4 Conclusion . . . 49

5 Experimental Results and system complexity . . . 50
5.1 Error Metrics . . . 51
5.1.1 Accuracy . . . 51
5.1.2 Precision & Recall . . . 51
5.1.3 F1 score . . . 52
5.2 Experimenal results . . . 53

5.2.1 Hyperparameters Vs parameters of a model . . . 53
5.2.2 HyRCoML Hyperparameters . . . 53
5.2.3 Achieved results and interpretation . . . 58
5.2.4 Estimated complexity . . . 60
5.2.4.1 Estimated complexity of the training process . . . 60
5.2.4.2 Estimated complexity in inference mode . . . 62
5.2.4.3 Estimated complexity and memory on STM32 . . . 63
5.2.5 Measured complexity by X-CUBE-AI . . . 63
5.2.5.1 X-CUBE-AI definition . . . 63
5.2.5.2 X-CUBE-AI features . . . 63
5.2.5.3 ONNX . . . 64
5.2.5.4 HyRCoML complexity analyses . . . 64
5.3 Conclusion . . . 66

General Conclusion . . . 67

Accepted Paper . . . 69

Bibliography . . . 74

List of Figures

1.1 STM around the world 4
1.2 STM customers worldwide 5
1.3 System for real-time ECG anomaly detection 7
1.4 AI projects management methodology [19] 8

2.1 ECG waveform segments in normal sinus rhythm[20] 11
2.2 Heart rhythm morphology[25] 12
2.3 Weather conditions[26] 13
2.4 Stock exchange[27] 14
2.5 Health monitoring[28] 14
2.6 Supervised anomaly detection[30] 15
2.7 Semi-supervised anomaly detection[31] 16
2.8 Semi-supervised anomaly detection[32] 16
2.9 Biological neuron (left) vs. artificial neuron (right)[34] 17
2.10 Artificial neural network[35] 18
2.11 One node from a feed forward neural network 19
2.12 Minimizing a loss function [36] 20
2.13 Recurrent neural network[37] 21

3.1 ECG signals in record 205 for both channels (above: MLII, below:V1)[39] 24
3.2 Modified limb lead II electrode placement[40] 24
3.3 100.hea file 25
3.4 ECG signal in STM dataset 26
3.5 Sliding window overlapping [45] 28
3.6 Sliding windows on ECG MIT BIH recordings 29
3.7 Sliding windows on ST custom recordings 30

4.1 Architecture overview of the proposed HyRCoML network 34
4.2 Simple illustration of a reservoir computing 35
4.3 Input and recurrent reservoir network topology 36
4.4 X_R indicates the number of incoming recurrent connections for each reservoir unit.X_I indicates the number of outgoing connections from each input unit 37
4.5 Unidirectional and bidirectional reservoir generated states 38
4.6 Cumulative and individual explained variances 39
4.7 One Class SVM 40
4.8 Illustration of OC-SVM hypersphere [62] 42
4.9 Decision boundary not very ‘curvy’ with small Gamma [63] 43
4.10 Decision boundary very ‘curvy’ with big Gamma [64] 43

4.11 LSTM Cell and It's Operations [65] 44
4.12 LSTM autoencoder architecture 45
4.13 Loss distribution on the test normal samples 46
4.14 Loss distribution on the test anomalous samples 46
4.15 Create the environment on anaconda 47
4.16 Install requirements 47
4.17 STM Kernel on Jupyter 48

5.1 Precision and recall graphical illustration 52
5.2 Confusion matrix representation[66] 52
5.3 Model parameters and hyperparameters[67] 53
5.4 Variance and cumulative variance over the 75 principle components 55
5.5 Decision boundary for One-Class SVM with a linear kernel 57
5.6 Decision boundary for One-Class SVM with a 3 degrees polynomial kernel 57
5.7 Decision boundary for One-Class SVM with a 5 degrees polynomial kernel 58
5.8 Decision boundary for One-Class SVM with a 5 degrees polynomial kernel 58
5.9 ONNX tool for AI models conversion[20] 64
5.10 HyRCoML and complexity analysis diagram 65
5.11 X-CUBE-AI results for PCA+OCSVM 66
5.12 X-CUBE-AI results for PCA+OCSVM 66

List of Tables

3.1 Composition of original MIT-BIH Vs derived dataset 30
3.2 Composition of original ST dataset Vs ST derived dataset 31

4.1 Hardware Characteristics 47

5.1 Results for record 100 when reservoir size is 100 54
5.2 Results for record 100 when reservoir size is 150 54
5.3 Results for record 115 when reservoir size is 100 54
5.4 Results for record 115 when reservoir size is 150 55
5.5 Variance for 45 principle components 56
5.6 Cumulative variance over 45 principle components 56
5.7 Measured performances with HyRCoML network 59
5.8 Measured performances with LSTM network 59
5.9 HyRCoML on derived ST dataset 60
5.10 Estimated complexity of training 62
5.11 Estimated complexity on inference mode 62
5.12 Estimated testing, training time and memory on STM32 63

List of acronyms

AI	*Artificial Intelligence*
ML	*Machine Learning*
DL	*Deep Learning*
ECG	*ElectroCardioGram*
CVD	*CardioVascular Disease*
RC	*Reservoir Computing*
PCA	*Principle Component Analysis*
OC-SVM	*One Class Support Vector Machine*
MCU	*Micro Controller Unit*
ANN	*Artificial Neural Network*
RNN	*Recurrent Neural Network*
LSTM	*Long Short Term Memory*
RRNN	*Recurrent Reservoir Neural Network*
MLII	*Modified limb Lead II*
ESN	*Echo State Network*
TS	*Time Series*
MACC	*Multiplication ACcumulation Computation*
MSE	*Mean Squared Error*

BPTT *Back Propagation Through Time*

WFDB *WaveForm DataBase*

API *Application Programming Interface*

HyRCoML *Hybrid Reservoir Computing Machine Learning*

OCC *One Class Classification*

ARM *Advanced RISC Machines*

CPU *Central Processing Unit*

ONNX *Open Neural Network eXchange*

General Introduction

Recently AI techniques have sent vast waves across healthcare, it's reshaping the health and medical services, and its use is becoming a reality in many medical fields and specialties. AI, ML, and DL enable healthcare stakeholders and medical professionals to identify healthcare needs and solutions faster with higher accuracy, using data patterns to make informed medical or business decisions quickly and efficiently.

The main AI use cases in healthcare include information analysis and collaboration, remote patient monitoring, and intelligent diagnostics and assistance [1]. The power of AI can be leveraged to augment physicians in their abilities to diagnose and treat patients, mitigate their own burnout and enhance the entire spectrum of the patient journey. Specifically, in cardiovascular medicine, AI-based systems have found new applications in cardiovascular imaging, cardiovascular risk prediction, and newer drug targets.

Cardiovascular diseases are considered as a leading cause of human death worldwide, with over 17 million people known to lose their lives annually due to CVDs [2], [3], [4]. An ECG is still one of the primary diagnostic tests for detecting cardiovascular abnormalities by measuring the electrical activity generated by the heart as it contracts [5]. However, to detect such early indicators for undiagnosed patients, it would be necessary to perform continuous health monitoring over long periods of time. So far, this has been impractical for several reasons. First, long-term surveillance generates large amounts of data, most of which are probably non-pathological. Also, searching such data for indicative segments is a tedious and error-prone task, which must be performed by qualified and trained medical professionals.

An advanced real-time anomaly detection model, to identify CVDs at their early stage, could effectively help to reduce the mortality rate by providing an instantly fast notification to provide better support. To automatically detect heart arrhythmia in an ECG stream, an algorithm must implicitly recognize the distinct wave types and discern the complex relationships between them over time. This is difficult to achieve by hand-crafted rules due to the variability in wave morphology among patients.

Hence, the goal of this internship is to leverage the simplicity of a reservoir recurrent neural network, in order to create a real-time online learning approach that uses a few bunches of data for learning to detect abnormalities within an ECG signal. For this approach to be broadly available, it should be implementable on a low-cost MCU.

This report contains a comprehensive summary of all of the tasks completed as part of my graduation project for an engineer's degree from the Higher School of Communications Tunis (SUP'COM). This study is divided into five chapters: We begin by introducing the general context of the project, namely the host organization and the scope of the project. Then we study the main theoretical background needed for a better and deeper understanding of the solution. In the next chapter, both datasets used to evaluate the solution are explained in depth. Then, we present the architecture of the proposed hybrid model. After that, comes the results section, where the performance and the complexity of the model were presented. And finally, we will wrap up our project with a conclusion and perspectives for future work.

Chapter 1

Internship context & Scope Definition

Introduction

This thesis was entirely realized during an internship at STMicroelectronics, Milan Italy, working as a member of the Artificial Intelligence Software & Tools System Research and Development team. The aim of this chapter is to introduce the framework this thesis was conceived in, the company and its philosophy as well as the team itself. Finally, it discusses the overall goal and structure of the thesis.

1.1 The host enterprise

In this section, we will present the host enterprise and its activity sector.

1.1.1 Enterprise presentation

STMicroelectronics is a global leader company, established in 1987, serving customers across the spectrum of electronics applications with innovative semiconductor solutions [6]. ST aims to be the undisputed leader in multimedia convergence and power applications leveraging its vast array of technologies, design expertise, and combination of an intellectual property portfolio, strategic partnerships, and manufacturing strength. Among semiconductor companies evaluated, STMicroelectronics is ranked the sixth-most admired company, in its first year. Additionally, it is by far the highest-ranking Europe-based chip maker on the list.

The Figure 1.1 describes the geographic spread of STMicroelectronics around the world.

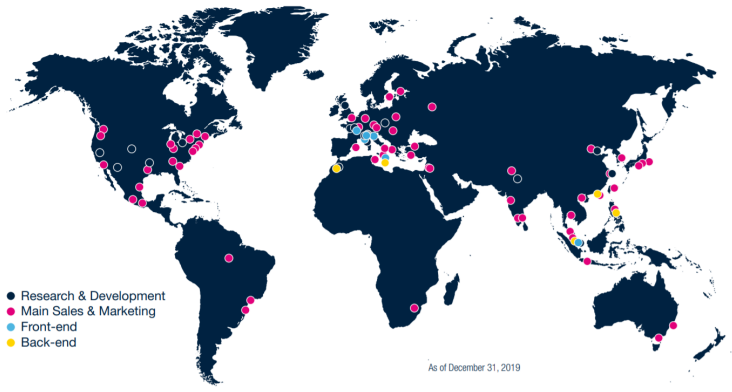


Figure 1.1: STM around the world

ST works with clients of all sizes, individuals, startups, and big companies. ST's clients are distributed all over the world, speaking more than 12 languages and covering more than 35 countries. Figure 1.2 illustrates ST's top-ranked clients.

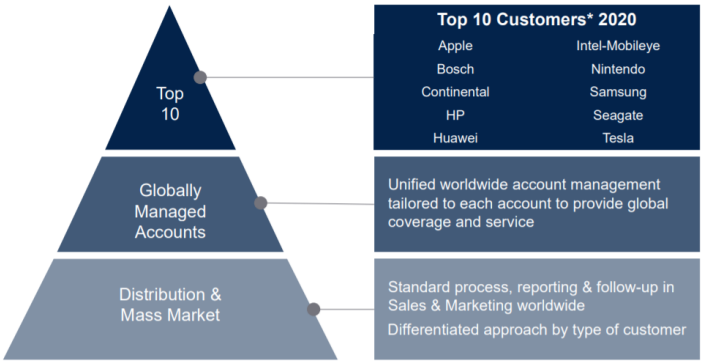


Figure 1.2: STM customers worldwide

1.1.2 Sector of activity

ST products are found everywhere, microelectronics are making a positive and innovative contribution to people's lives: from energy management and savings to trust and data security, from healthcare and wellness to smart consumer devices, in the home, car and office, at work and at play [7].

- Smart Mobility: The first trend is the shift from traditional cars to smarter mobility solutions: with connected, digital, electric vehicles and their supporting infrastructure moving from a niche to high volumes globally. Smart Mobility means thinking differently about how we move around, reducing pollution and congestion. ST's car electrification solutions enable makers to build better more affordable electric vehicles while our ADAS solutions support our customers' quest to make driving safer for drivers and pedestrians alike.
- Power & Energy: The second trend is the need for much more efficient power and energy management across all devices and systems and is key to address the increasing global energy demand while reducing environmental impacts. ST has a long history of innovations in the field of power and energy management, and this is recognized now, more than ever, the world needs to become a better steward of the resources it uses. ST was the inventor of smart power technologies and it continues to lead the movement to green technologies that enables the industry to save energy without compromising innovation.
- 5G & IoT: The third trend is the large-scale deployment of the Internet of Things (IoT), accelerated by the rollout of 5G connectivity. This trend is transforming every area of our lives where we work, where we live, the cars we drive, and the objects we use with billions of cloud-connected devices for personal, corporate, and public applications. ST is at the forefront of data security with solutions that help the customers implement the latest cryptographic measures.

1.1.3 STMicroelectronics, Milan Italy

STMicroelectronics is located in Milan Italy and is part of the semiconductor and other electronic component manufacturing industry. It employs 6,000 staff, the Milan

facilities match Grenoble in importance. Agrate Brianza employs around 4,000 staff and is a historical base of the company. The site has several fab lines (including a 300 mm (12 in) fab) and an RD center. Castelletto employs 300 to 400 staff and hosts some divisions and RD centers.ST Italy has acquired its strength and credibility through hundreds of internationally diverse projects allowing it to be one step ahead of its competitors.

1.2 Thesis overview and problem statement

In this section, we will provide the motivation and objectives of our work. We will also provide a general overview to figure out the current problem and the motivation to implement a new and optimized solution.

1.2.1 Problem Statement & Motivation

Cardiovascular diseases are among the main causes of premature deaths worldwide [8]. Some pathological conditions can be diagnosed at an early stage by electrocardiography (ECG), which measures the electrical excitation of the heart. In such cases, quick interventions may lead to better outcomes and even save lives [9] [10]. However, to detect such early indicators in undiagnosed patients it would be necessary to carry out continuous health monitoring over extended periods of time. Until now this has been impractical for several reasons. First of all, long-term surveillance generates large amounts of data, most of which are probably non-pathological. Also, searching such data for indicative segments is a tedious and error-prone task, which must be performed by qualified and trained medical professionals. Also, one of the major challenges to detect anomalies is set by the consideration that a normal heartbeat varies with age, body size, activity, and emotions [11].
Automated ECG arrhythmia detection has received much interest in the literature. Typical approaches involve neural networks[12] [13] [14], support vector machines [15], fuzzy cognitive maps [16] or the extraction and analysis of a priory defined features [17]. While many of these algorithms achieve detection accuracy of beyond 90 percent, they are generally not applicable for ultra-low-power hardware implementation.

Therefore, personalized real-time online learning, that uses a limited and fixed amount of data for training, and is implementable on MCU is mandatory.

1.2.2 Objectives & Proposed solution

The main objective of this work consists of proposing a scalable, personalized, always-on system architecture for continuous signal processing and online learning of anomalies (see Figure 1.3). Signal Input should be evaluated continuously and in real-time. No segmentation of the signal or synchronization to heartbeats is required, and for this matter, a sliding window will be introduced among the binary data. Furthermore, the learning process should only use a few bunches of data, the minimum amount that achieves adequate performances. Moreover, to make such an approach broadly available,

it should be implementable on a low-cost MCU that usually features limited computational power (in the order of tens of MHz) and limited memory (hundreds of Kbytes) in 10 to 100s µW/MHz power consumption envelope.
To automatically detect heart arrhythmia's in an ECG stream, an algorithm must implicitly recognize the distinct wave types and discern the complex relationships between them over time.

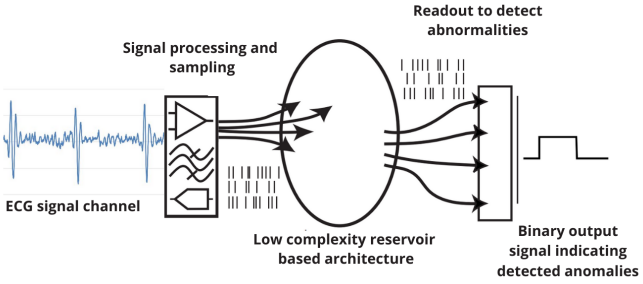


Figure 1.3: System for real-time ECG anomaly detection

In this work, we focus on recurrent reservoir neural networks (RRNNs), which first processes sequentially the time series ECG data with a dynamically learned model using a limited amount of input data. RC is a family of NNs whose recurrent weights are set randomly, and they are kept fixed at inference time [18]. Despite this drastic simplification, which requires neither backpropagation nor stochastic gradient descent learning, as well as multiple epoch runs, RC provides a rich pool of dynamic features that are suitable for solving a large variety of tasks. Leveraging it, we propose an unsupervised procedure for RRNN online training by using limited in size, anomaly-free time-series data, using the four modules pipelined architecture: RC module, representation module, dimensionality reduction module, followed by a readout module. In this study, we compare another RNN architecture implemented to achieve the same objective to prove that the proposed topology is dramatically faster than other models and achieves competitive accuracy. The proposed approach has been evaluated on the publicly available MIT-BIH dataset, also modified to mimic the temporal sliding of input tensors on the ECG stream data.

1.3 Project management approach

The work approach plays a major role in project realization and success. That's why we have to choose a methodology that responds to the project requirements and makes the work process more efficient.

Considering the nature of our project which is based on artificial intelligence and coding development, data preparation, modeling, evaluation, and complexity analysis. A customized way dedicated to manage the AI projects imposes itself to make the project a success.

1.3.1 Meetings & team work

We collaborate on this project online with my supervisor and co-supervisor within STMicroelectronics company. In our working process we had two types of meetings :

- Status Update Meetings:

Duration: 1 hour.
This meeting is held once a week between the project team members. In this meeting, I need to present the work that I've completed and discuss how we might enhance the past work and what are the future steps to reach the desired goal. Quality information and inspiring interactions are raised throughout the meeting's period. It's about ensuring that we are moving in the right direction, and we can stick to completing the upcoming steps with consideration of other team members' feedbacks.

- Communicate progress for close deadlines Meetings:

Duration: 30 minutes.
If a deadline is approaching and something is urgent, it needs to be dealt with as soon as possible. This meeting is held daily before an approaching deadline. The meeting's main advantage is that it is fast and efficient. The aim of this meeting is to take hard decisions if the target is to be reached soon, to manage schedules and prioritize well the tasks, in order to deliver the work on time. In case of unexpected problems, we need to find a quick and reliable solution for it.

1.3.2 AI projects management methodology description

In this section, we will present the different steps to manage an AI project. The method is divided into 6 stages as Figure 1.4 shows:

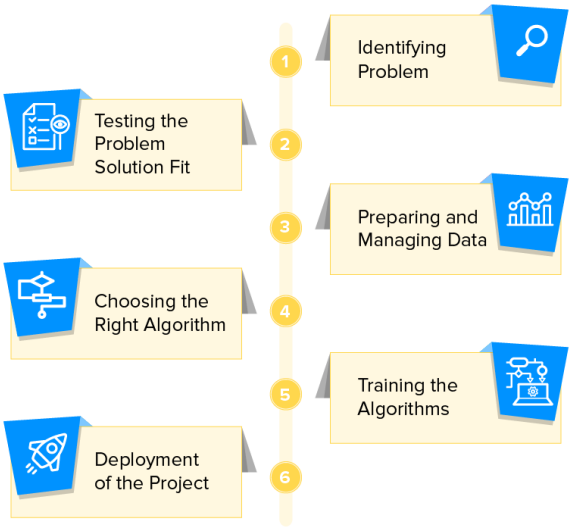


Figure 1.4: AI projects management methodology [19]

- Identifying Problem:
  Defining the problem is the very first step for us when it comes to managing AI projects. We start by asking two questions: What are we interested in solving? and How will we solve it?

- Testing the Problem Solution Fit:
  This stage should provide guidance on how to initiate an AI project. Before we begin the AI project development process, we need to use a variety of methodologies to verify the problem-solution fit, including the classic lean methodology and the Product Design Sprint.

- Preparing and Managing Data:
  When we reach the point where we made sure that there exists a customer AI solution for the project and we have the confidence that the AI is implementable, we start gathering data and handling their management, organizing and cleaning up the data, and add labels if needed.

- Choosing the Right Algorithm:
  We should be aware that there are different types of algorithms, which vary on the basis of the learning we need.

- Training the Algorithms:
  After we've chosen an algorithm, we'll proceed to train the model, which entails feeding data into the model while retaining the accuracy of the model in consideration.

- Deployment of the Project:
  Deployment means making an application ready to be delivered. The most appropriate platform is chosen based on the needs of the customer.

1.4 Conclusion

Through this chapter, we highlighted the context of our project by presenting in the first place the host company shortly and its activities. We introduced the problem and the currently available solutions and we identified the objectives and contributions of this project. This chapter also presented the specification and the required work to realize the proposed solution, detailing the meeting planning, the project life cycle, and the time tasks planning.

Chapter 2

Background Theory

Introduction

This chapter summarizes the relevant fundamentals for further understanding of the topic of the proposed work. We will expose briefly the basic concepts of ECG signal, anomaly detection process, and neural network technology to understand which type of models are the most suitable to fit the project's specification.

2.1 ECG signal

An ECG, also known as an EKG or electrocardiogram, is a test used to detect and reports the intensity and timing of the electrical activity in the heart using electrodes placed on the skin. This data is shown on a graph that depicts each phase of the electrical signal activity as it passes through the heart. There are three main components to an ECG: the P wave, which represents the depolarization of the atria; the QRS complex, which represents the depolarization of the ventricles; and the T wave, which represents the repolarization of the ventricles as shown in Figure 2.1.

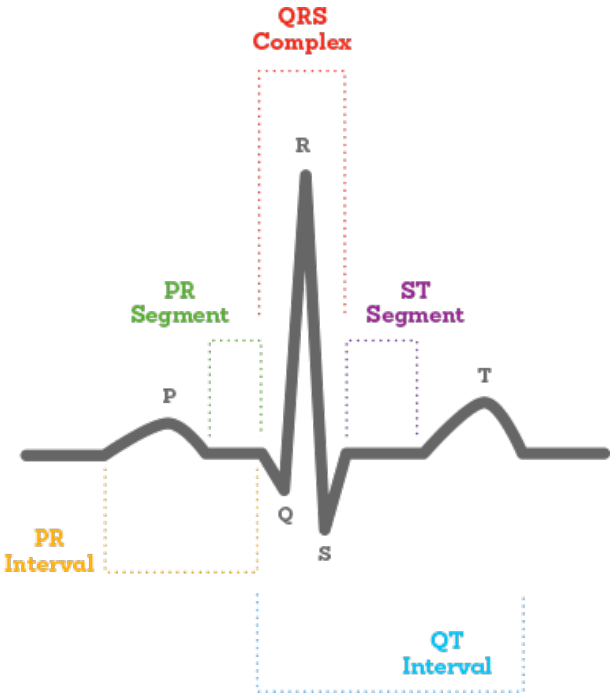


Figure 2.1: ECG waveform segments in normal sinus rhythm[20]

Electrocardiogram (ECG) analysis is the most frequent clinical cardiac examination, which is a powerful detection technique for a variety of cardiovascular issues since it is affordable, easy, and pain-free. As a result, ECG analysis has been extensively studied during the previous two decades [21]. Mostly, since an ECG signal depicts the electrophysiological activities that correspond with the sequence of depolarization and repolarization of the atria and ventricles [22], it is a key indicator for heart functional investigation.

The drawback of ECG analysis is that the heartbeats of each user feature have a specific

morphology, which also depends on the wearable electrode channel and its position [23]. Also, one of the challenges in ECG anomaly detection, that there is no specific pattern for an ECG signal as it is a complex signal, and the heartbeats change drastically over time.

2.2 Arrhythmia

2.2.1 What is Arrhythmia

Arrhythmia is abnormal changes in the heart rate due to incorrect heart beating which results in failure in the blood pumping. Arrhythmias can occur at any age [24]. Some arrhythmias have no symptoms, while others are dramatically debilitating and can even lead to sudden cardiac death. The common arrhythmias symptom is premature beats, palpitations or skipped beats, dizziness, fatigue, light-headedness, and fainting. The abnormal electrical activity of the heart can be life threatening.

2.2.2 Types of arrhythmia

There are several types of arrhythmias and each type has a pattern that possible to identify its type. There are two major categories of arrhythmias. The first category consists of arrhythmias formed by a single irregular heartbeat, herein called morphological arrhythmia. The other category consists of arrhythmias formed by a set of irregular heartbeats, herein called rhythmic arrhythmias.
The abnormal electrical activity of the heart can be life-threatening. People with hypertension, diabetes or coronary artery disease are more likely to experience arrhythmias.

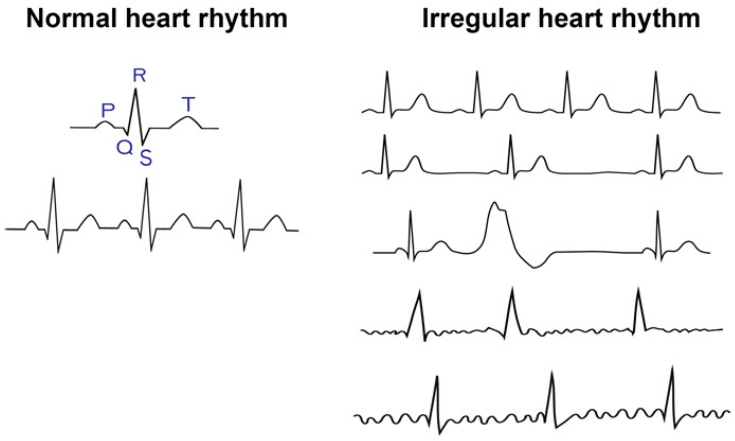


Figure 2.2: Heart rhythm morphology[25]

2.3 Time series data

2.3.1 Definition

Time series data is a collection of observations obtained through repeated measurements over time. While plotting the points of a time series data on a graph, we can see that one of the axes is always time. Time series data can be found everywhere, as time is a principal component of everything observable. Sensors and devices are continuously producing a constant flow of time series data while our world becomes more and more automated. Such data can be used in a variety of businesses.
Time series data can be classified into different types:

- Periodic time series data, the measurements are gathered at regular time intervals also known as metrics (Exp health monitoring).
- Non-periodic time series data, the measurements gathered at irregular time intervals also known as events (Exp logs collection).
- Linear time series data, when a data point at time t, can be viewed as a linear combination of past or future values.
- Non-linear time series data, they have features that cannot be modeled by linear equations: time-changing variance, asymmetric cycles, higher-moment structures...
- Seasonal time series data, they are identified when the data has regular and predictable patterns at time intervals.

2.3.2 Time series examples

Time series data may be used for a variety of purposes, including:

- Weather data can be tracked on a daily, hourly, or weekly basis as shown in Figure 2.2.

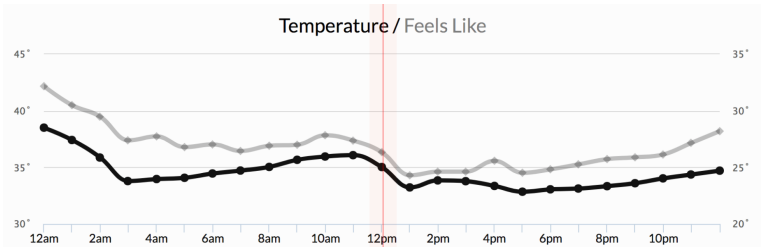


Figure 2.3: Weather conditions[26]

- Stock market analysis and time series prediction.
  A time series is a type of investment that monitors the movement of stocks data points over time, such as the price of an asset, with data points collected at regular intervals as shown in Figure 2.3.

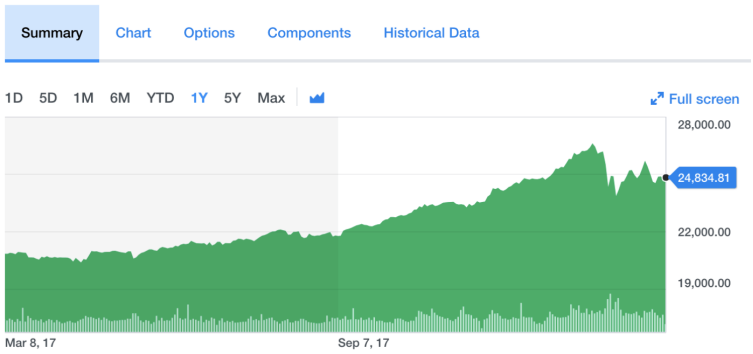


Figure 2.4: Stock exchange[27]

- Medical equipment that allow you to see your vitals in real time.
  Patients' health real time-monitoring, such as an electrocardiogram (see Figure 2.4), is another well-known example of time series data. An ECG analyzes the heart's rhythm to determine if it is functioning normally or not.

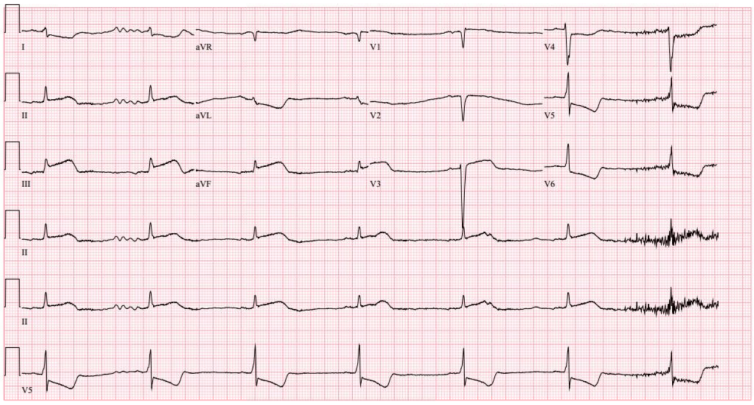


Figure 2.5: Health monitoring[28]

2.4 Anomaly Detection

2.4.1 Definition

Anomaly detection also known as outlier detection, is a data mining phase that identifies rare data points, events, items, and observations that differ from the expected behavior of a dataset, which means raise suspicions because of the significant difference from the majority of the data [29]. Typically, anomalous items might reveal important occurrences, such as a technological failure, bank fraud, a structural defect, medical problems, or errors in a text. In enterprise IT, anomaly detection is commonly used for:
-Data cleaning.
-Systems health monitoring.
-Intrusion detection.
-Event detection in sensor networks.
-Fraud detection.

2.4.2 Anomaly detection with ML

Machine learning is progressively being used to automate anomaly detection. But, it can only work under certain conditions.
Anomaly detection can occur under three different settings, Let's look at each setting in depth.

2.4.2.1 Supervised learning of anomalies

The training data is labeled with "normal" or "abnormal".
The supervised setting is the ideal setting. It is the instance when a dataset comes neatly prepared for the data scientist with all data points labeled as anomaly or normal. In this scenario, all anomalous points are known before. In this case, simple classifier can be trained, and applied. However, this configuration is irrelevant, because we need to know in advance all the anomalies and label the data correctly. In many cases the anomaly may appear as new during the testing phase. The visual representation is available in Figure 2.6.



Figure 2.6: Supervised anomaly detection[30]

2.4.2.2 Semi-supervised Anomaly Detection

In this kind of setting, all data points are considered as "normal", and it is contaminated with "anomaly" points.
The clean setting is the less-ideal case where a bunch of data is presented to the model, and all data are presumed to be nominal data points. Then, it is up to the modeler to detect the anomalies inside of this dataset. The concept is that a typical class model has already been taught, and anomalies may be discovered by deviating from the learned model. This method is sometimes referred to as "one-class" categorization. One-class SVMs and autoencoders are well-known methods used for this approach. The visual representation is available in Figure 2.7.

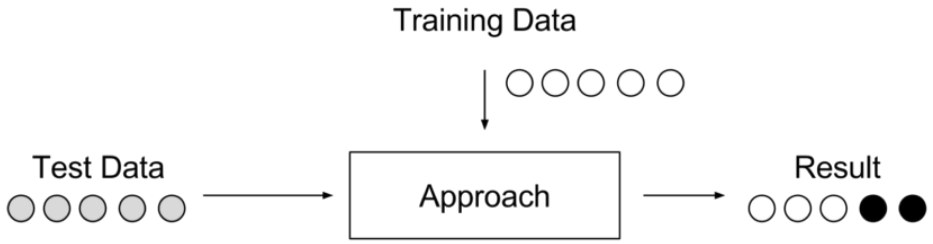


Figure 2.7: Semi-supervised anomaly detection[31]

2.4.2.3 Unsupervised learning of anomalies

In this unsupervised settings, the training data is unlabeled and consists of "normal" and "anomaly" points.
The datasets in the unsupervised scenario do not have their parts labeled as nominal or anomalous. There is no ground truth from which to expect the outcome to be. The model must decide by itself what is anomalous and what is nominal. The idea is that unsupervised anomaly detection methods grade data only on natural characteristics of the dataset. Distances or densities are commonly used to determine what is typical and what is an outlier. The visual representation is available in Figure 2.8.

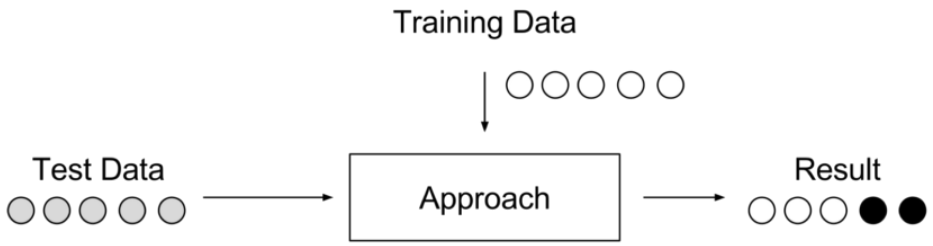


Figure 2.8: Semi-supervised anomaly detection[32]

2.5 On-line learning

Online learning covers methods that update their models after observing a new event and can immediately serve predictions based on the updated model. In other words, it is a type of machine learning that utilizes a dynamic input where it takes data in real-time and applies it to a machine learning algorithm without memorizing it. It is usually used when; it is unnecessary to save and memorize the data set or when the data set is a function of time [33]. Compared to the traditional ML methods, online learning is a fundamentally different and unique approach, it embraces the fact that learning environments can change from second to second, so it takes the state of the environment at each time step into consideration.

2.6 Artificial Neural Network

In the disciplines of AI, machine learning, and deep learning, neural networks mimic the activity of the human brain, allowing computer programs to identify patterns and solve complex problems.

2.6.1 Definition of an ANN

Artificial neural networks are a technology, based on brain and nervous system research. These networks mimic the biological neural networks system, however, they only employ a subset of features from the biological neural networks.
The main components of a neural network are neurons. A single neuron has several inputs called dendrites coming from other preceding neurons. The neuron processes the input and if a certain action potential is reached, the neuron "fires" through its single output called the axon. The output of the axon will be forwarded to all or some of the following connected neurons.
An artificial neuron represents the biological neuron in a simplified way as shown in Figure 2.5.

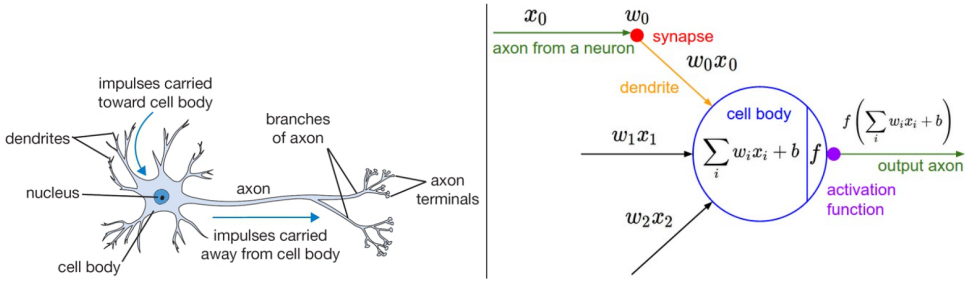


Figure 2.9: Biological neuron (left) vs. artificial neuron (right)[34]

ANNs consist of an input layer, hidden layers, and an output layer as illustrated in Figure 2.6, such that each layer is composed of a precise number of neurons that are linked by one or more to the neurons of the hidden layer. By firing activation functions, input layer nodes transfer information to hidden layer nodes, while hidden layer nodes either fire or remain dormant based on the evidence provided.

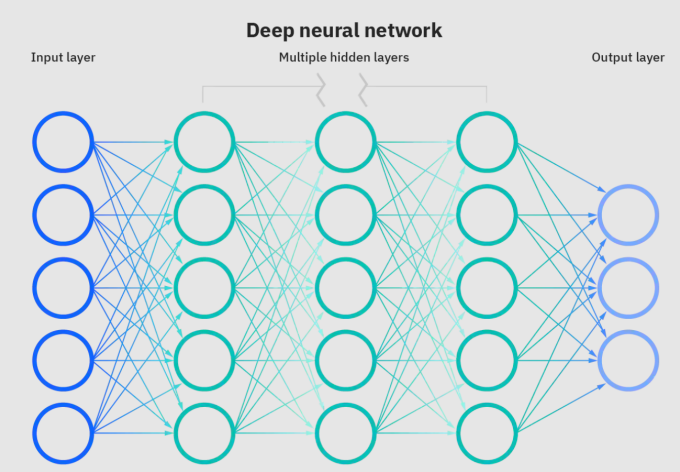


Figure 2.10: Artificial neural network[35]

Neural networks rely on training data to learn and improve the performances over time. However, once these learning algorithms are fine-tuned for accuracy, they become a powerful tool in computer science and AI, allowing us to quickly categorize and classify data at a high velocity.

2.6.2 How do Neural Networks work

A neural network is composed of several layers. Each layer serves a specific purpose, and the more layers we have, the more complicated the network becomes.
The simplest form of a neural network is composed of three layers:

- The input layer: This layer picks up the input signals and transfers them to the next layer. It gathers data from the outside world.
- The hidden layer: This layer performs all the back-end tasks of calculation.
- The output layer: This layer transmits the final result of the hidden layer's calculation.

Let's think of each individual node as an independent linear regression model, composed of input data, weights, a bias, that provides an output. The formula would look something like equation (1):

$$\sum_{i=1}^{n} w_i * x_i + bias = w_1 * x_1 + w_2 * x_2 + w_3 * x_3 + bias \qquad (1)$$

Once an input layer is determined, weights are assigned. These weights are used to determine the relevance of each variable, with larger ones contributing more to the output than smaller ones. After that, all of the inputs are multiplied by their respective weights and then summed together. The output is then processed through an activation function to determine the output. The activation function serves to apply some kind of non-linear function on top of the summation to break linearity within the network, see Figure 2.7.

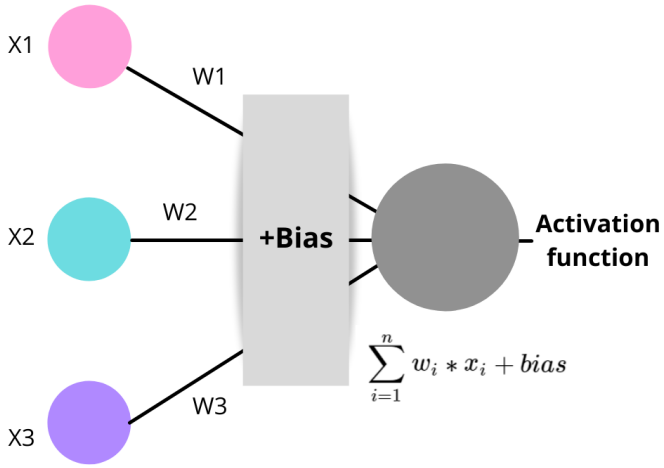


Figure 2.11: One node from a feed forward neural network

The only thing left to define before talking about the training process is the Loss function.
A loss function is a function that tells, how well the neural network is performing for a certain task. The intuitive way to do it is to take each training batch, pass it through the network and compare the output with the actual label that we want to get from the network by calculating the loss function.
The most popular loss function is the mean square error (MSE) presented in the above formula, Where y stands for the number we want to get from the network, and y with a hat is the number we actually got by feeding our example through the network (2):

$$L(Y, \hat{Y}) = \frac{1}{m} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2 \qquad (2)$$

If the Loss function is big then the network isn't performing very well, we want the loss to be as small as possible to make sure that the network is learning.
The problem of training is equivalent to the problem of minimizing the loss function. There are a lot of algorithms that optimize functions the loss function that can be used for training a neural network. These algorithms can be gradient-based or not, in the sense that they are not only using the information provided by the function, but also by its gradient. After applying an optimization function, we should update the weights, and then the output is back-propagated to minimize error loss until the algorithm converges to its minimum.
With each training example, the parameters of the model adjust to gradually converge to the minimum as shown in Figure 2.8.

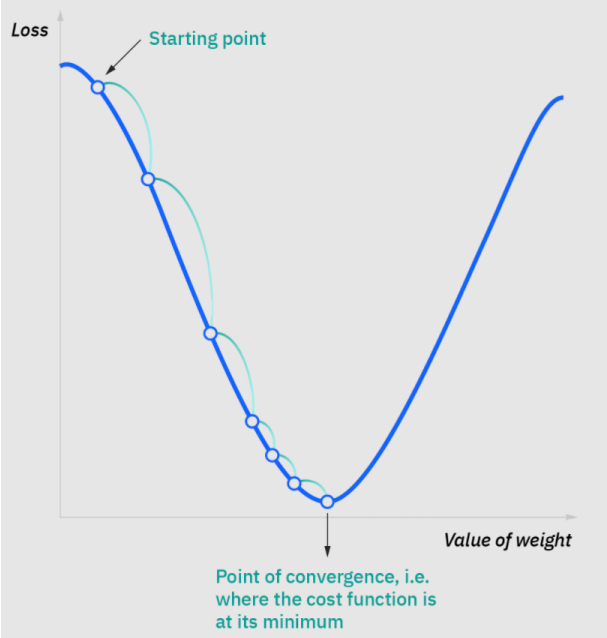


Figure 2.12: Minimizing a loss function [36]

2.7 Recurrent Neural Network

A recurrent neural network is a type of artificial neural network which uses sequential data or time-series data. These deep learning algorithms are commonly used for ordinal or temporal problems, such as language translation, natural language processing, signal processing, and speech recognition. Like other neural networks, recurrent neural networks need training data to learn from it. The main difference is that they are distinguished by their "memory" as they take information from prior inputs to influence the current input and output.
While traditional deep neural networks assume that inputs and outputs are independent of each other, the output of recurrent neural networks depends on the prior elements within the sequence. In this network, the output of a layer is saved and transferred back to the input. This way, the nodes of a particular layer remember some information about the past steps, Figure 2.9 illustrates a visual of the RNN. The combination of the input layer is the product of the sum of weights and features. Here, each node remembers some of the information of its antecedent step. The model retains some information from each iteration, which it can use later. The system self-learns when its outcome is wrong. It then uses that information to increase the accuracy of its prediction in back-propagation.

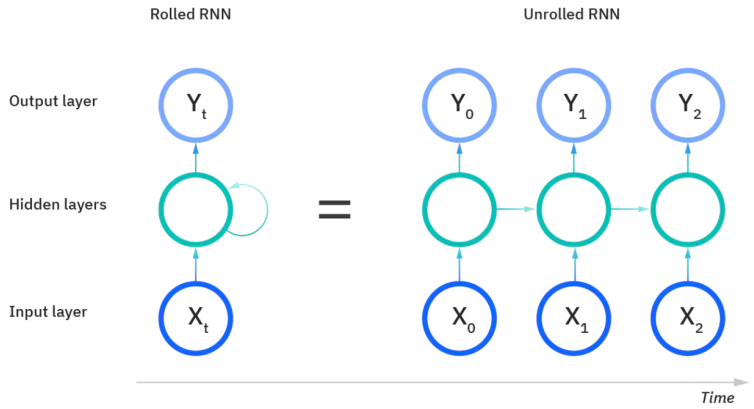


Figure 2.13: Recurrent neural network[37]

To estimate the gradients, recurrent neural networks use backpropagation through time algorithm BTTP, which is significantly different from standard backpropagation because it is particular to sequence data. Traditional backpropagation uses the same concepts as BPTT, in which the model trains itself by computing errors from its output layer to its input layer. These computations enable us to correctly adjust and fit the model's parameters. BPTT differs from the other approaches because it sums errors at each time step whereas normal networks do not need to sum errors because they do not share parameters between layers.
RNNs commonly encounter two issues throughout the BTTP process: exploding gradients and vanishing gradients. The magnitude of the gradient, which is the slope of the loss function along the error curve, defines these difficulties. When the gradient is too tiny, it gets smaller and smaller, updating the weight parameters until they are insignificant like zero. When this happens, the algorithm stops learning. When a gradient is too big, it explodes, resulting in an unstable model. The model weights will get excessively huge in this scenario. One of the proposed solutions to these issues is to reduce the number of hidden layers within the neural network, to reduce the complexity of the network, or use LSTM architecture that will be defined in chapter3.

2.8 Conclusion

Through this chapter, we have clarified the different concepts that will help us to better undertake the project and understand the architecture. The next chapter will cover the details about the proposed topology to implement our solution.

Chapter 3

Datasets understanding and preparation process

Introduction

The following chapter presents the different datasets used to evaluate the performances of this work and the preparation process for these datasets. This aspect of the solution is very important, due to the fact that most of the implemented features are going to revolve around the data we prepared. At first, we will define the MIT BIH dataset and the STMicroelectronics customed dataset. Then, a description of the methods and tools used to re-annotate the data will be provided.

3.1 Datasets

3.1.1 MIT BIH arrhythmia database

The MIT-BIH Database [38] was the first set of standard test material that was publicly available to evaluate any arrhythmia detectors and it has been used for that purpose as well as for basic research into cardiac dynamics at about 500 sites worldwide since 1980. This database consists of 48 half-hour excerpts of two-channel ambulatory ECG recordings, obtained from 47 subjects studied by the BIH Arrhythmia Laboratory between 1975 and 1979. These 48 half-hour excerpts were split into two groups:

- 23 (the "100 series name") were chosen at random from a collection of over 4000 24-hour ambulatory ECG recordings collected from a mixed population.
- 25 (the "200 series name") were selected to include examples of uncommon but clinically important arrhythmias that would not be well represented in a small random sample.

The first group is intended to serve as a representative sample of the variety of wave-forms and artifacts that an arrhythmia detector might encounter in routine clinical use. A table of random numbers was used to select tapes, and then to select half-hour segments of them. Segments selected in this way were excluded only if neither of the two ECG channels was of adequate quality for analysis by human experts.
Records in the second group were chosen to include complex ventricular, junctional, and supraventricular arrhythmias and conduction abnormalities. Several of these records were selected because features of the rhythm, QRS morphology variation, or signal quality may be expected to present significant difficulty to arrhythmia detectors; these records have gained considerable notoriety among database users.
The subjects were 25 men aged 32 to 89 years, and 22 women aged 23 to 89 years so a variety of ages is represented by this database. (Records 201 and 202 came from the same male subject.)
The recordings were digitized at 360 Hz for each channel with 11-bit resolution over a 10-mV range.
An example can be seen in Figure3.1, in which ten seconds from record 205 with two ECG channels and their annotations are represented.

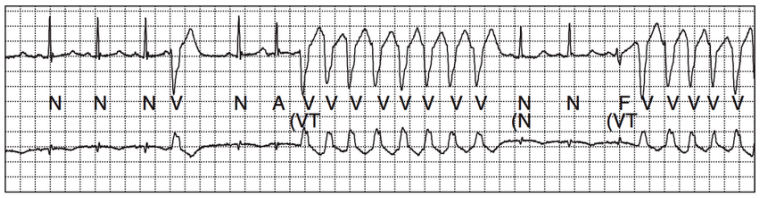


Figure 3.1: ECG signals in record 205 for both channels (above: MLII, below:V1)[39]

3.1.1.1 ECG lead configuration

In most of the records, the upper signal is a modified limb lead II (MLII), obtained by placing the electrodes on the chest as illustrated in Figure3.2. The lower signal is usually a modified lead V1 (occasionally V2 or V5, and in one instance V4); as for the upper signal, the electrodes are also placed on the chest. This configuration is routinely used by the BIH Arrhythmia Laboratory. Normal QRS complexes are usually prominent in the upper signal.
A notable exception is record 114, for which the signals were reversed. Since this happens occasionally in clinical practice, arrhythmia detectors should be equipped to deal with this situation. In records 102 and 104, it was not possible to use modified lead II because of surgical dressings on the patients; modified lead V5 was used for the upper signal in these records.

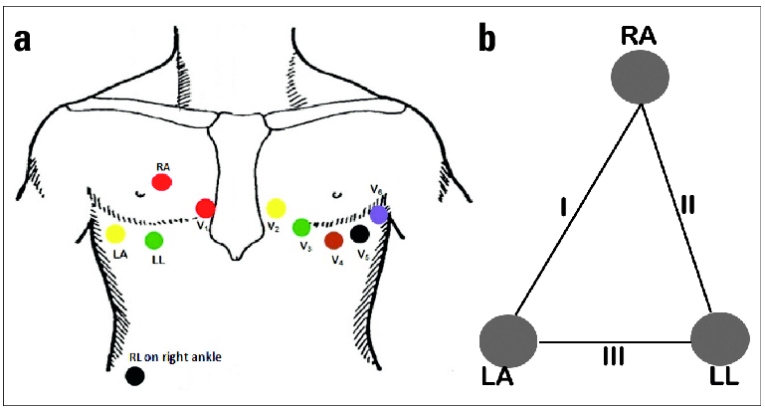


Figure 3.2: Modified limb lead II electrode placement[40]

In this work, the MLII channel was used, and only normal rhythms and the five most prevalent anomaly types were considered, which accounts for 95.6% of total recording time.

3.1.1.2 MIT BIH annotation process

An initial set of beat labels was produced by a simple slope-sensitive QRS detector, which marked each detected event as a normal beat. For each record, the charts were given to two cardiologists, who worked on them independently. The cardiologists added additional beat labels where the detector missed beats, deleted false detections as necessary, and changed the labels for all abnormal beats. They also added rhythm labels, signal quality labels, and comments [41].

Once both sets of cardiologists' annotations for a given record had been transcribed and verified, they were automatically compared beat-by-beat, and another chart recording was printed. This chart showed the cardiologists' annotations in the margin, with all discrepancies highlighted. Each discrepancy was reviewed and resolved by consensus. The corrections were transcribed, and the annotations were then analyzed by an auditing program, which checked them for consistency and which located the ten longest and shortest R-R intervals in each record (to identify possible missing or falsely detected beats).
The MIT BIH database is composed by:

- The header file (.hea): is a short text file that describes the contents of the signals (including the name of the signal file, number of samples, signal format, type of signal, and detailed clinical information as shown in Figure 3.3.

Record 100 (MLII, V5; male, age 69)

Medications: Aldomet, Inderal

Beats	Before 5:00	After 5:00	Total
Normal	367	1872	2239
APC	4	29	33
PVC	-	1	1
Total	371	1902	2273

Supraventricular ectopy

- 33 isolated beats

Rhythm	Rate	Episodes	Duration
Normal sinus rhythm	70-89	1	30:06

Signal quality	Episodes	Duration
Both clean	1	30:06

Points of interest:

11:03 Normal sinus rhythm
25:13 PVC
26:09 APCs
27:55 Normal sinus rhythm

Figure 3.3: 100.hea file

- The binary file (.dat): Almost all records include a binary file (.dat) containing samples of one or more digitized signals stored in 212 format. These files store the waveforms, but they cannot be interpreted properly without their corresponding header files.
- Attribute file (.atr): Annotation files are binary files, contain sets of labels (labels that generally refer to specific samples in associated signal files), each of which describes a feature of one or more signals at a specified time in the record. Annotation files should be read with their associated header files.

3.1.2 STM custom dataset

This dataset was provided by STMicroelectronics company. It is composed of one record for a single person, segmented and annotated. This dataset consists of 3 leads ECG signal digitized with 128 Hz, hence there are three samples for each time slot and each sample is stored in 2 bytes, check Figure 3.4. This record contains both normal and abnormal heartbeats, also noisy and clean heartbeats.

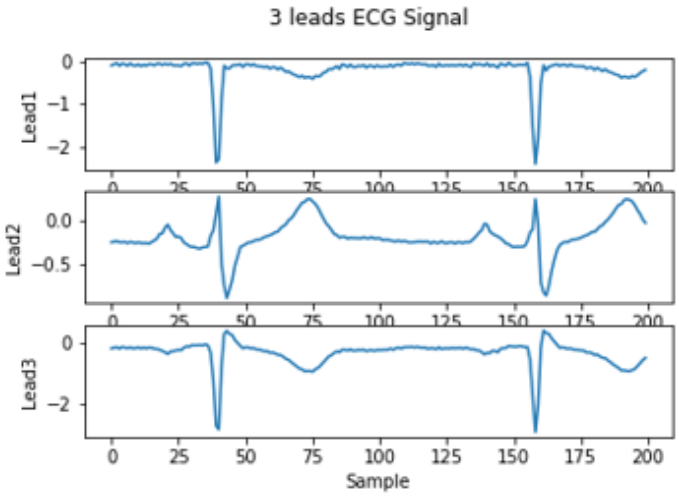


Figure 3.4: ECG signal in STM dataset

This dataset consists of two files:

- x0001_20210309.acea which is the annotation file.

Let's start to explore the annotation which is an xml file format. This file is composed by several sections:
-Section1:
<Section id="16" name="Record information" itemsCount="6">
<Info id="25" name="Sampling Rate" value="128"/>
<Info id="26" name="Amplitude Resolution [count/mV]" value="1000"/>

This section contains general information about the recording like the sampling rate which is 128 Hz, date and time of the recording, the amplitude resolution, number of samples per lead which is equal to 7561216, the number of leads and the patient id.

-Section 2:
<Section id="11" name="Beats" itemsCount="48504">
<Beat id="101" name="N" pos="39">
<Info id="21" name="TemplateId" value="1"/>
</Beat>
<Beat id="101" name="N" pos="158">
<Info id="21" name="TemplateId" value="1"/>
<Info id="22" name="RR" value="930"/>

</Beat>

This section contains all the detected heartbeats, 48504 detected heartbeats in this recording. And we have the position for each heartbeat in the DataStream. The first detected heartbeat is in the 39th sample and the second one is in the 158th position… It also contains information about the Heartbeat rate HR and other detailed information that isn't needed for this work.

-Section 3:
<Section id="12" name="Rhythm annotations" itemsCount="219">
<Rhythm id="218" name="ST Elevation" start="15360" end="19200">
<Info id="20" name="max Elevation" value="121 uV"/>
<Info id="20" name="HR max" value="75 bpm"/>
<Info id="20" name="average HR" value="75 bpm"/>
<Info id="24" name="Lead" value="Unknown2"/>
</Rhythm>

This section contains the classification of anomalous beats, all the beats not inserted in this section are 'normal'. Only heartbeats with anomalies are mentioned. For example, from sample 15360 to sample 19200 these samples belong to anomalous heartbeats. We have 219 [start, end] intervals. Other information that we won't need are mentioned in this section like anomaly type, HR, lead...

-Section 4:
<Section id="13" name="Noise annotations" itemsCount="93">
This section contains the intervals where noise is too high and prevents the system from detecting something. These noisy parts are removed from the signal file (an external algorithm should deal with noisy parts of the signal)
The remaining 3 sections are useless for this kind of work.

- x0001_20210309.ecg which is the signal file.

Let's move now to the ECG file, which contains the raw data in binary format. It is an ISHNE file format.
An ISHNE Hotler Standard Output File Format is a single file structurally organized in a header followed by a large data block containing all stored electrocardiographic digital samples. The format of this file is the outcome of several meetings with different manufacturers and it aims to maximize the ratio between simplicity and flexibility [42].

3.2 Data preparation and modification

3.2.1 Intuition and requirements

Since we are considering time series streaming-like data, the location of the anomaly cannot be accurately determined unless using a priory segmentation algorithm. Heartbeat's segmentation implies a priory knowledge in time of the anomaly to split the ECG recording into individual heartbeats. In the literature, there were numerous QRS detection methods proposed for the segmentation of ECG signals [43] [44]. Generally, most of the foregoing methods exhibited good performance in QRS detection; however, issues relating to power efficiency and implementation complexity are the main concerns for the MCU system. Therefore, we also modified the dataset and introduced a sliding window over the time series ECG data, that generated multiple input shapes to be detected as normal or abnormal by the proposed pipeline. These were used for both the learning and testing phases.

3.2.2 Sliding window

Because the data in Streaming applications is continuous, it won't be practical to wait for all of it to be streamed before starting the processing. Of course, we can evaluate each incoming event as it arrives and go on to the next, but there may be times when we need to aggregate the data. In such circumstances, we must establish a window and perform processing on the data contained within it.
A sliding window slides over the stream of data according to a specified interval such that it's possible for the windows to overlap. As a result, an event can belong to more than one sliding window. In fact, the sliding window gives a smoother aggregation over the incoming stream of data since it's not jumping from one set of input to the next set, rather it's sliding over the incoming stream of data as shown in Figure 3.5.
It's possible to configure a Sliding window also to slide based on time or by the count of events.



Figure 3.5: Sliding window overlapping [45]

3.2.3 Modified dataset with sliding windows

In the MIT BIH arrhythmia database and ST custom dataset, most beat labels were placed around the R-wave peak. So working with these datasets needs an a-priory exact

segmentation of the heartbeats which means a priory knowledge of the precise position of the heartbeat. Unfortunately, the complexity of this segmentation step is unknown and therefore cannot be assumed implementable on the MCU. Therefore, in this work, a non-segmented ECG signal will be used for the online training and the database will be modified as follows, shaping more realistic processing conditions.

3.2.3.1 Derived MIT BIH database

For the MIT BIH recordings, We applied a sliding window of size 300 samples, which corresponds approximately to the number of samples per heartbeat.
Assuming a healthy heart and a typical rate of 70 to 75 beats per minute, each cardiac cycle, or heartbeat, takes about 0.8 seconds to complete the cycle [46].
The sampling frequency for the MIT BIH dataset is 360, so we can estimate the number of samples per heartbeat equal to 360*0.8 300 samples. This explains the chosen size of the sliding window. This fixed-size window will slide over the recordings with a stride equal, e.g., to 50 samples and so an overlapping of 250 samples. To be as realistic and generic as possible, the starting point of the window should be different from the first sample in the recording as we cannot guarantee the exact beginning of the heartbeat in a non-segmented real application. Figure 3.6 shows an example of the waveform samples per consecutive sliding window.

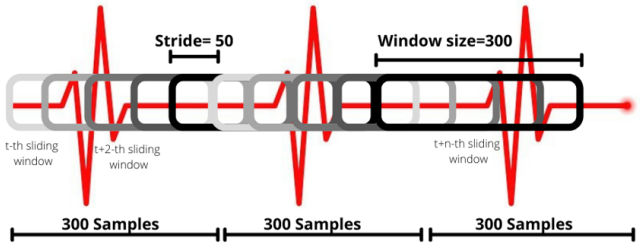


Figure 3.6: Sliding windows on ECG MIT BIH recordings

3.2.3.2 Derived ST custom dataset

Same as for the MIT BIH database, We applied a sliding window of size 100 samples over the ST custom dataset, which corresponds approximately to the number of samples per heartbeat.
Assuming a healthy heart and a typical rate of 70 to 75 beats per minute, each cardiac cycle, or heartbeat, takes about 0.8 seconds to complete the cycle.
The sampling frequency for the ST custom dataset is 128, so we can estimate the number of samples per heartbeat equal to 128*0.8 100 samples on average. This explains the chosen size of the sliding window for this dataset. This fixed-size window will slide over the recordings with a stride equal to, e.g. 30 samples and so an overlapping of 70 samples. To be as realistic and generic as possible, the starting point of the window should be different from the first sample in the recording as we cannot guarantee the exact beginning of

the heartbeat in a non-segmented real application. Figure 3.7 shows an example of the waveform samples per consecutive sliding window.

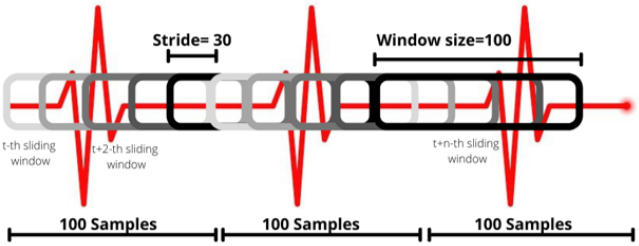


Figure 3.7: Sliding windows on ST custom recordings

3.2.3.3 Reannotation process

The training process needs only to use a limited amount of abnormal-free data to learn normal patterns from them. However, the model needs to identify the deviations to an expected normal pattern that are presented in the training data. In the labeling process, we reannotated the recordings of both datasets as we are not working on the original segmented heartbeats. For each window, if the number of samples that belong to an anomalous heartbeat exceeds 50% of the window size, the derived window will be considered anomalous, if it's less than 50% the derived window is considered normal. Equally, we applied the sliding window on the testing set using the same window size and same stride as for the training set. The testing set contains both normal and abnormal heartbeats, following the same reannotation procedure.
A numerical description of the original and derived MIT BIH dataset is shown in Table 3.1.

	Recording numbers	**Segmented MIT BIH dataset:** Stride=300 Window size=300 Starting point=0	**Derived dataset:** Stride=50 Window size=300 Starting point= (105,68,219,272)
Shape of training set	100 205	(100,300,1) (100,300,1)	(100,300,1) (100,300,1)
Shape of testing set	100 205	(2155, 300, 1) (2522, 300, 1)	(13414, 300, 1) (15132, 300, 1)
Number of normal data	100 205	2126 2457	13240 15226
Number of abnormal data	100 205	29 65	174 390

Table 3.1: Composition of original MIT-BIH Vs derived dataset

	STM dataset	**Derived STM dataset** (window size= 100 Stride=30 Starting point=22)
Sampling frequency	128 Hz	128 Hz
Shape of training set	(300,100,1)	(300,100,1)
Accuracy	0.741	0.7113
F1 score	0.823	0.7963
Training time (ms)	747	768
Testing time (ms)	47096	42693

Table 3.2: Composition of original ST dataset Vs ST derived dataset

3.2.4 Programming tools

To implement the sliding window, to read and prepare the data that we have conceived, we used a wide range of technologies and libraries that we list here:

- WFDB

The native Python waveform-database (WFDB) package is a library of tools for reading, writing, and processing WFDB signals and annotations used to read and process the MIT BIH database without any dependencies on the original WFDB software package.
This package is a large set of well-tested, interoperable command-line tools and functions (subroutines) for reading and writing files in the formats used by PhysioBank databases [47].

- The ElementTree XML API

The xml.etree.ElementTree module in python implements a simple and efficient API for parsing and creating XML data.
XML is an inherently hierarchical data format, and the most natural way to represent it is with a tree. ElementTree has two classes for this purpose - ElementTree represents the whole XML document as a tree, and Element represents a single node in this tree. Interactions with the whole document (reading and writing to/from files) are usually done on the ElementTree level. Interactions with a single XML element and its sub-elements are done on the Element level [48].

- ISHNE Holter Library for Python

A library to work with ISHNE-formatted Holter ECG files as the binary ECG signal file in the ST custom dataset is in ISHNE format [49] [50].

3.3 Conclusion

In this chapter, we presented in detail the different datasets that we worked on, and the process behind the data preparation and modification through the exportation of the project requirements. We also specified the different tools used to implement the solution.

Chapter 4

Proposed network architecture

Introduction

The theoretical studies and data preparation procedure carried out in the previous chapters are present together to establish an architecture that meets the functional requirements of the desired solution. In this chapter, we present the topology of the invented HyRCoML network and a deep LSTM architecture that can serve as a benchmark for comparison purposes. We will also go through the tools and modules that were used to implement the project.

4.1 HyRCoML topology

Dealing with time-series stream data has been tackled with a large variety of methodologies and applied to a wide range of scenarios [51]. Common in machine learning is to express the classifier as a combination of an encoding and a decoding function. The encoder generates a compact representation of the input, while the decoder is a discriminative model that computes the output given the encoder representation. An encoder based on RNN [52] is particularly suitable to process time-series data.

The proposed HyRCoML architecture is shown in Figure 4.1. It's a hybrid topology that combines both neural network and machine learning technologies.
HyRCoML is composed of four modules [53]: first a reservoir, then a representation module followed by a dimensionality reduction, and finally a readout module that we will see in detail in the next sections.

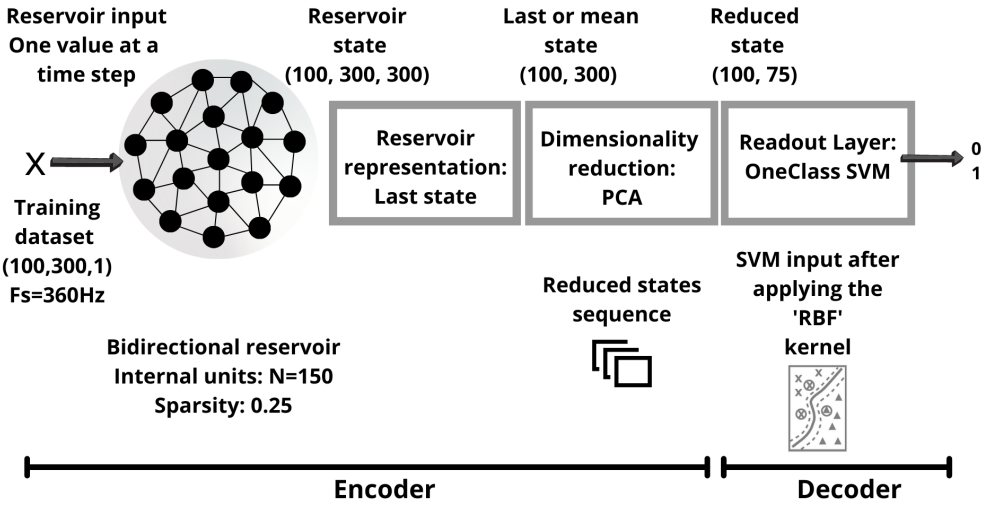


Figure 4.1: Architecture overview of the proposed HyRCoML network

This approach is based on a paradigm of recurrent reservoir computing network RC. The choice was based on the fact that RC is one of the simplest recurrent neural networks to process time-series data in terms of temporal and spatial complexity. This is

really important as our system requires simplicity in training and inference process to be implementable on MCUs.
RC is a family of NNs whose recurrent weights are set randomly, and they are kept fixed at inference time, it efficiently generates a representation of the input TS that can be further processed by machine learning classifiers. Often RC is a part of an RNN with sparsely connected neurons. The connectivity is fixed, and weights are randomly assigned at initialization time, without requiring further learning. This topology is particularly interesting because the only weights that are modified during training are for the synapses that connect the hidden neurons to output neurons so that the network can approximate specific temporal patterns.

4.1.1 Reservoir module

To avoid any costly operation associated with backpropagating through time, and to avoid any problem related to exploding gradients and vanishing gradients that we may encounter while using a traditional recurrent network as seen in chapter2, the RC approach took a totally different direction. RC operates on a random, large, fixed, recurring neural network that processes a sequence of inputs signals that are dependent on each other i.e. time dependencies, and induces a nonlinear response signal in each neuron within this "reservoir". The connections in the reservoir are assigned once and are completely random and kept untrained (W_{in}, W_r).
These are input weights and recurrent connections weights, respectively. The reservoir weights do not get trained, input neurons are connected to the reservoir and feed the input activations into the reservoir, these too are assigned untrained random weights. This relies on the idea that a big random expansion of the input vector can often make it easy for a model to fit the data. In a reservoir, all weights would be fixed at random which expands the input vector, and then using that expanded representation we try and fit the readout module. In Figure 4.2, a reservoir computing is presented in its simplest and more general way to show how it generates a rich pool of heterogeneous dynamics useful to solve many different tasks.

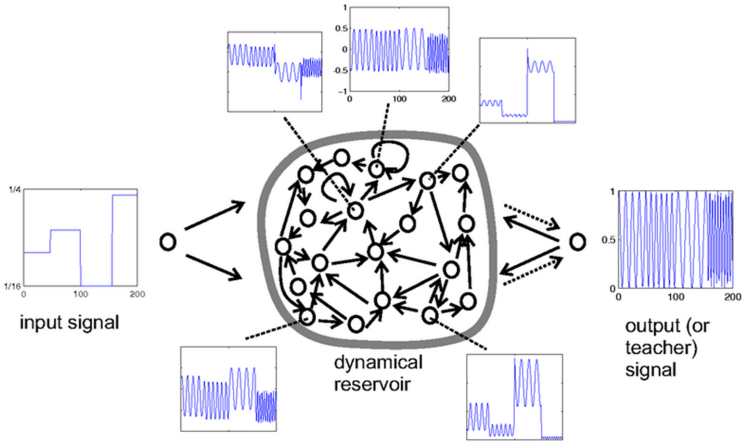


Figure 4.2: Simple illustration of a reservoir computing

A detailed illustration of the implemented reservoir computing for this project is shown in Figure 4.3. The time series ECG signal is with T=300 time instants and the observation at time t is denoted as $x(t) \in \mathcal{R}$. The recurrent reservoir is governed by the state-update equation (3):

$$h(t) = tanh(W_{in}x(t) + W_r h(t-1)) \qquad (3)$$

where h(t) is the RC state at time t that depends on its previous value h(t − 1) and the current input x(t).

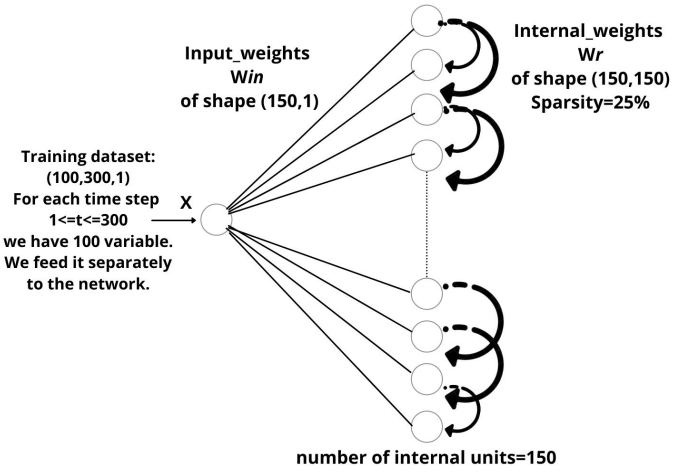


Figure 4.3: Input and recurrent reservoir network topology

4.1.1.1 Sparcity

In both machine learning and neuroscience, sparse neural networks, in which only a few neurons or connections are active, are frequent. In machine learning, on the other hand, ”sparseness” is associated with a penalty term that effectively causes certain connection weights to become so tiny or zero.
The main purpose of a reservoir computing is to create a pool of randomly and sparsely connected recurrent neurons. For the proposed network, the number of units within the reservoir is 150 neurons, with 25% connectivity over the recurrent weights.
Making the connections among reservoir neurons sparse has the fundamental practical advantage to reduce the cost of the reservoir state update operations [54]. Furthermore, having sparse connections between neurons is not only advantageous in computational terms as it enables fast sparse matrix computations but also can be beneficial to obtain a better performance in practice [55].
An illustration of sparsity in input to the reservoir and recurrent reservoir connections is presented in Figure 4.4.

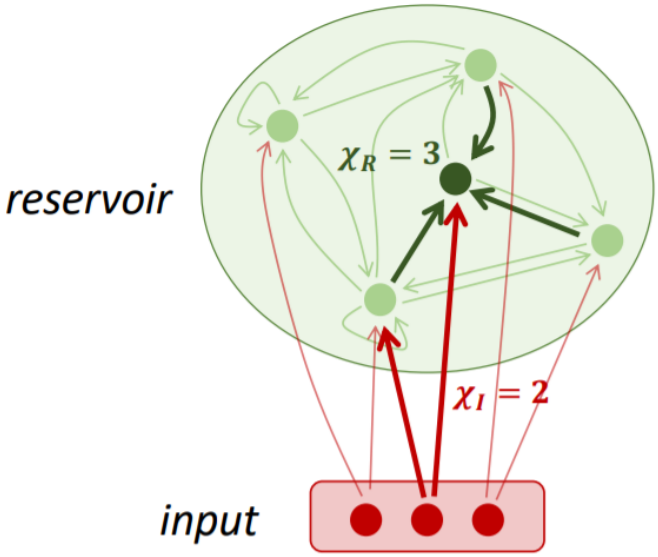


Figure 4.4: X_R indicates the number of incoming recurrent connections for each reservoir unit.X_I indicates the number of outgoing connections from each input unit

4.1.1.2 Bidirectionality

The model features a bidirectional reservoir since used in the context of time series prediction to incorporate both past and future information, to mitigate any short-term memory limitation, and to improve the accuracy of the model [56]. Bidirectional architectures have been successfully applied in RNNs to extract temporal features from the time series that accounts also for dependencies very far in time [57].
Bidirectionality is implemented by feeding into the same reservoir an input sequence both in direct and reverse order as shown in equations (4)(5):

$$\vec{h(t)} = tanh(W_{in}x(t) + W_r h(\vec{t-1})) \qquad (4)$$

$$\overleftarrow{h(t)} = tanh(W_{in}x(t) + W_r \overleftarrow{h(t-1)}) \qquad (5)$$

Where $\overleftarrow{x(t)} = x(\vec{T-t})$. The full reservoir state is obtained by concatenating the two state vectors for each input x(t) which are $\vec{h(t)}$ and $\overleftarrow{h(t)}$. This can capture longer time dependencies by summarizing at every step both recent and past information.

In this context, the whole time series is given at once and, thus, a bidirectional reservoir can be exploited in both training and test to generate a better representation of the input, check Figure 4.5 to see the difference between a unidirectional and bidirectional reservoir.

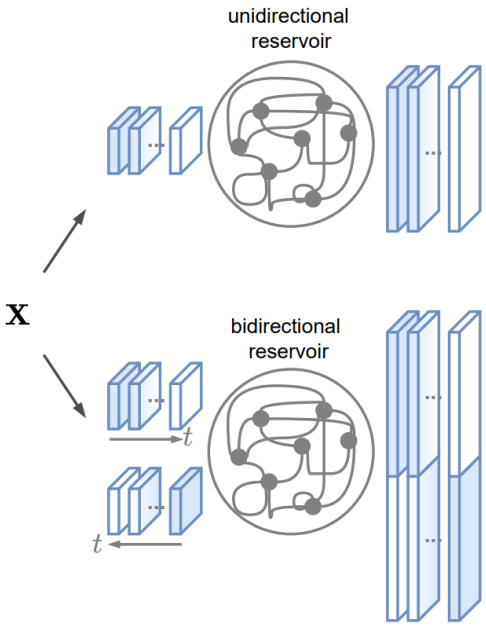


Figure 4.5: Unidirectional and bidirectional reservoir generated states

4.1.2 Representation module

This recurrent reservoir generates a representation of the input R_x that has a large size and embodies temporal features dependencies. After the whole input x is processed by the reservoir, the last state at time T, maintains scarce information about the first and past states thanks to the bidirectionality. Also, the input can be represented by the mean state which is the mean of all reservoir states.

$$R_x = h(t) \qquad (6)$$

4.1.3 Dimensionality reduction module

The dimensionality reduction module consists of projecting input representation from a high to a lower-dimensional space, usually relying on unsupervised criteria. Jointly with the RC, and in the context of reducing the dimension, a principle component analysis machine learning algorithm has been used. It projects data on the d eigenvectors of the covariance matrix. PCA has been already successfully applied to the reservoir network states, improving its prediction and modeling capabilities [58]. When coping with the massive sequence of generated reservoir states by the TS input, dimensionality reduction is only applied to a single sequence of the reservoir: last generated state or mean state, which reduces the complexity of the proposed topology.

Let's provide a simple way of computing PCA of the last state matrix X.
$X = (x_1 x_2 ... x_{100})$ is of shape $(M, N) = (100, 300)$ such that $M = 100$ is the number of observations and $N = 300$ is the number of samples per heartbeat.

The PCA consists of the following steps:

1. Calculate the mean vector (7) of each heartbeat representation.

$$mean = \frac{1}{M}\sum_{i=1}^{M}(y_i) \qquad (7)$$

2. Compute the covariance matrix (8), as shown in the following equation:

$$\text{Cov} = \frac{1}{M-1}\sum_{i=1}^{M}(y_i - mean)^T(y_i - mean) \qquad (8)$$

3. Calculate the eigenvectors and eigenvalues of the covariance matrix (9). The eigenvalues are λ_i and eigenvectors are ei.

$$\text{Cov}.e_i = \lambda_i.e_i, \quad i = 1, ...N \qquad (9)$$

4. Choosing components and forming a feature vector. The eigenvector with the highest value is the principal component. Then, the eigenvectors are ordered by eigenvalues from highest to lowest, which returns the components in order of significance. Subsequently, the dimensionality is reduced by selecting K=75 principal components that retain the physiological information [59].

5. Construct a projection matrix W from the "top" K eigenvectors. 6. Transform the 300-dimensional input dataset X using the projection matrix W to obtain the new 75-dimensional feature subspace (10).

$$X_{PCA} = XW \qquad (10)$$

Given an input representation of shape (100,300), by using the computed projection matrix, we can now transform the original data space onto the PCA subspace (the principal components subspace) obtaining a reduced input representation of shape (100,75). So we reduced the dimensionality of our dataset by compressing it onto a new feature lower dimension subspace, we only selected the subspace that contains most of the information hence a higher variance as shown in Figure 4.6.

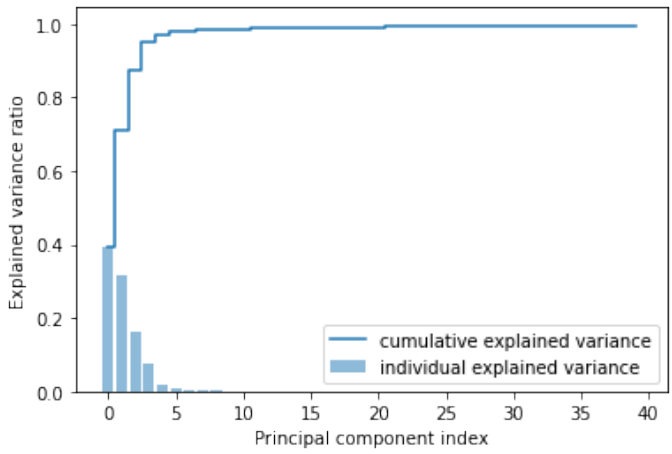


Figure 4.6: Cumulative and individual explained variances

The resulting plot indicates that the first principal component alone accounts for approximately 40% of the variance. Also, we can see that the first two principal components combined explain almost 75% of the variance in the dataset.

4.1.4 Readout OC-SVM module

The goal of machine learning classification models is to distinguish test data between a number of classes greater than two, using training data. However, in some cases, we may have only one class in the training dataset and the goal is to test new data and found out whether it is like the training data or not. This problem is known as anomaly detection problem where we would like to determine whether new test data is a member of a specific class, determined by our training data, or it's an outlier see Figure 4.7.

Similar to the current work, the ECG signal of the user won't be labeled in real-life applications, the stream data will be treated as normal in the training process and a search of anomalies will take place in the prediction process. One-Class Support Vector Machine (OC-SVM) is one of the state-of-the-art approaches for anomaly detection [60] due to their flexibility in fitting complex nonlinear boundaries between normal and novel data.

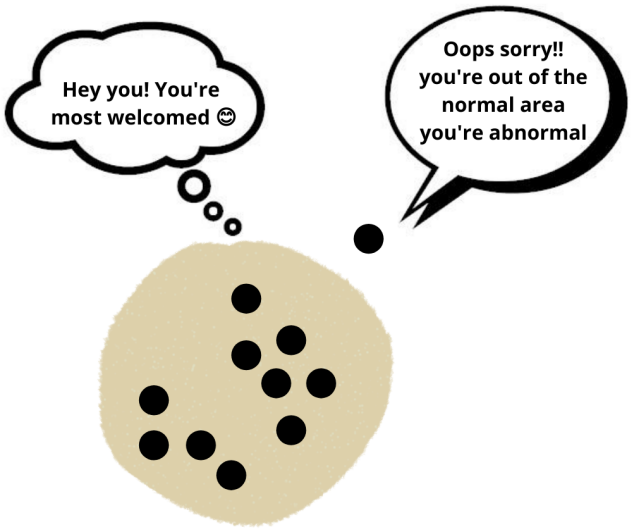


Figure 4.7: One Class SVM

4.1.4.1 OC-SVM for Imbalanced Data

Outliers are both uncommon and rare. Rarity indicates that they occur seldom in comparison to non-outlier data (so-called inliers). They don't fit nicely into the data distribution if they're unusual. Outliers may cause chaos on a data set. For example, an outlier far from the mass of instances in a single variable might distort summary statistics like the mean and variance.
In this project, we are dealing with imbalanced data. An anomaly within an ECG signal is unusual and each anomaly type has its specific pattern that is also different from one

person to the other. Anomaly identification is the process of finding outliers in a dataset, where the outliers are called "anomalies" and the rest of the data is called "normal."
In machine learning, one approach to tackling the problem of anomaly detection is one-class classification. OCC involves fitting a model on the "normal" data and predicting whether new data is normal or an outlier/anomaly.

4.1.4.2 Basic concepts of One Class Support Vector Machines

The studied encoder (Reservoir+Representation+PCA) seen in the previous sections generates a reduced representation of the ECG input signal, in order to map these states to the desired output by the readout module.
Usually, the readout layer of a reservoir network exhibits linear functionality and is trained by solving a convex optimization problem. Different from the classic linear readout, this topology identifies the unusual patterns (anomalies) by processing the temporal features of the generated state using the One-Class SVM approach.

Let's dive into the details of OCSVM [61]:
Assume we have a training dataset composed of N data objects, $x_i, \quad i = 1, .., N$, the goal of OCSVM is to compute a hypersphere able to encompass all of the instances in the dataset. This hypersphere is represented by center a and radius R which is the distance between the center and (any support vector) on the boundary of the hypersphere as shown in Figure 4.8. To find the best hypersphere that fits our data we need to minimize the error function L. This loss function should take into consideration two facts: the volume of the hypersphere and the distance from the boundary of any outlier object.
The target solution can be constrained with the requirement that almost all the presented data must be within the hypersphere, which is hard to get. To permit the possibility of the existence of outliers in the training set, the distance from a sample x_i to the center a can be strictly higher than the radius R. Therefore, to create this soft margin we should introduce an extra parameter C that will tune the trade-off between the volume of the hypersphere and the loss as well as slack variables ξ_i which measure the distance to the boundary when a certain object is outside the description.
The minimization problem is as follows (11):

$$\min_{R,a} R^2 + C \sum_{i=1}^{n} \xi_i \qquad (11)$$

$$\text{Subject to } ||x_i - a||^2 \leq R^2 + \xi_i \text{ for all } i = 1, ...n$$

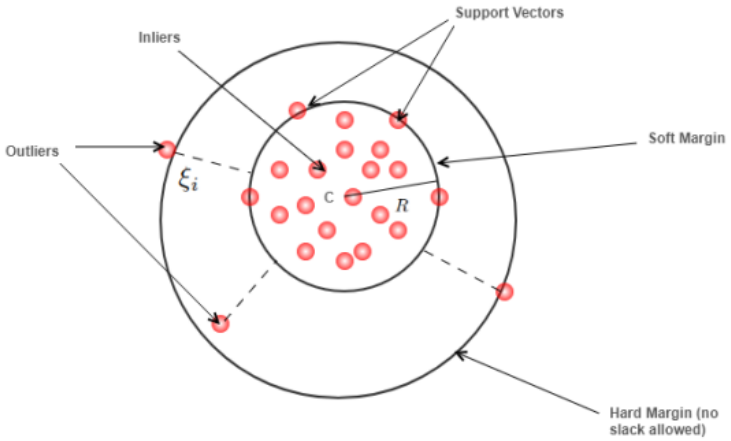


Figure 4.8: Illustration of OC-SVM hypersphere [62]

The parameter C is used to finetune the trade-off between overfitting and generalization. C is a positive and unbounded parameter which makes it hard to interpret, for this matter $\nu = \frac{A+B}{C}$ was introduced such that A and B are two constants. The new parameter ν is in the range $[0, 1]$ and it makes the regularization more interpretable. ν is upper bounded by the fraction of outliers and lower bounded by the fraction of support vectors e.g. if $\nu = 0.2$, at maximum 20% of the training samples are allowed to be considered as outliers and to be wrongly classified by the hypersphere. Consequently, at least 20% of the training samples should act as support vectors.

4.1.4.3 Kernel function

Generally, while dealing with complex data we use a kernel function before applying OCSVM on the original data. The kernel function takes the input data in its lower dimension space and computes the dot product of the transformed vectors in a higher-dimensional space. The kernel function $K(x, y)$ gives more flexibility to the model and gives better fitting descriptions. In this project, we chose to work with the 'RBF' kernel presented in (12):

$$(x \cdot l) \rightarrow K(x, l) = exp(-\frac{||x-l||^2}{\gamma^2}) \qquad (12)$$

But with the introduction of the kernel function, another parameter called the kernel-width parameter Gamma γ needs to be introduced as well. This parameter can be considered as the spread of the kernel which implies the decision region. If γ is too low, the decision boundary is less curvy resulting in a relatively large decision zone. If γ is high, then the decision boundary is curvier that may result in multiple decision boundaries around the data point. The choice of such parameter is so important and delicate, as a bad choice can result in overfitting if gamma is too high. A concrete example of the influence of Gamma on the decision boundary can be found in Figure 4.9 and Figure 4.10.

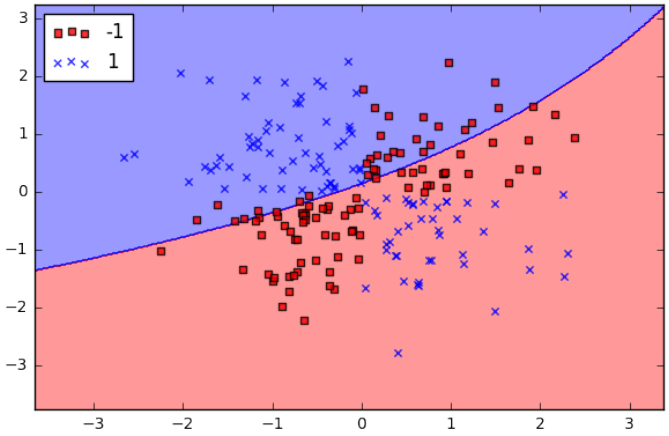


Figure 4.9: Decision boundary not very 'curvy' with small Gamma [63]

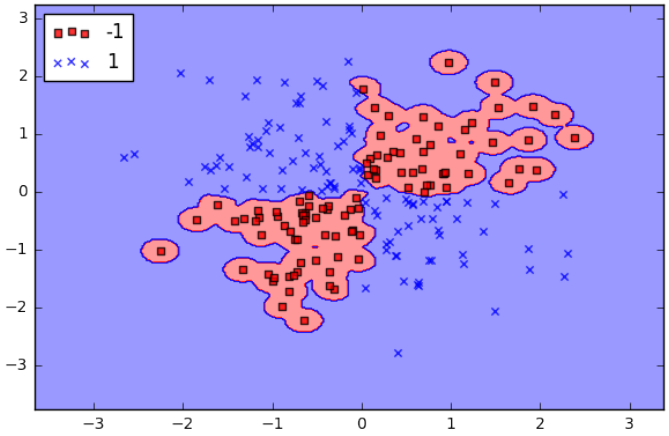


Figure 4.10: Decision boundary very 'curvy' with big Gamma [64]

For an input data of shape (100,75) which is the shape of the representation of the last reservoir state, we can assume having 100 observation each of dimension 75. So the dimension of the input state is 75. The 'RBF' kernel is going to compute new data $f_1, f_2, f_3 ... f_{100}$, from the used training data $x_1, x_2, x_3, ..., x_{75}$, such that (13) :

$$f_1, f_2, f_3 ... f_{100} = K(x_1, x_2, x_3, ..., x_{75}) \qquad (13)$$

And K is the 'RBF' kernel function that generates a new input of shape (100,100).

4.2 LSTM based architecture

This architecture is based on a fully trainable RNN implementing LSTM cells. The main purpose behind implementing such an architecture is to use it as a benchmark for comparison purposes with the HyRCoML in terms of performances and model complexity.

4.2.1 LSTM overview

RNNs have a fundamental flaw known as vanishing gradient that was mentioned in the 2nd chapter, which means they have trouble learning long dependencies between entities, more precisely they cannot maintain information between entities that are several steps apart. RNNs can forget what they have seen in long sequences, so they have short-term memory which is a big problem. LSTMs are a special kind of RNNs that have the capability to learn long-term dependencies. LSTMs are created as a solution for this problem and they are intended to prevent the problem of long-term dependency. An LSTM cell is represented in Figure 4.11.

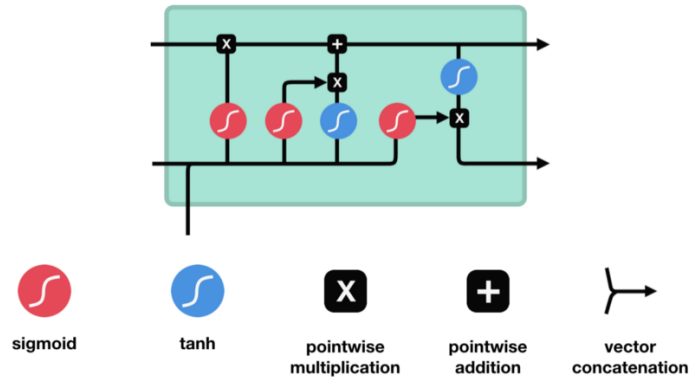


Figure 4.11: LSTM Cell and It's Operations [65]

The main concept of an LSTM network is the cell state and the different gates. The cell state acts as a transportation highway that transfers the needed information to the sequence chain. We can consider it as the "memory" of the network. In principle, the cell state can carry meaningful information throughout the sequence's processing. As a result, information from earlier time steps might make its way to later time steps, decreasing the short-term memory effects. Information is kept or deleted from the cell state via gates as the cell state travels. The gates are neural networks that determine whether an information about the cell state is permitted or not. During the training, the gates might learn what information is important to preserve and what information to forget or to delete. LSTM presents 3 types of gates: A forget gate, input gate, and output gate.

4.2.2 LSTM autoencoder benchmark architecture

An LSTM Autoencoder is an implementation of an autoencoder for sequence data using an Encoder-Decoder LSTM architecture. An encoder-decoder LSTM is set up to receive the input sequence of a given dataset, encode it and produce a compressed representation of the input, then decode it, and reproduce it. The model's ability to reproduce the input sequence is used to evaluate its performance. The architecture of the used LSTM autoencoder is shown in Figure 4.12, the encoder is composed of 2 unidirectional stateless LSTM layers of size 256 and 128 respectively. The encoder is going to encode the data by compressing it to a lower-dimensional space. The decoder as well is composed

of 2 unidirectional stateless LSTM layers of size 128 and 256 as well, it will reconstruct the input from the compressed representation. An error function will measure the loss of information between the original input and the reconstructed one.

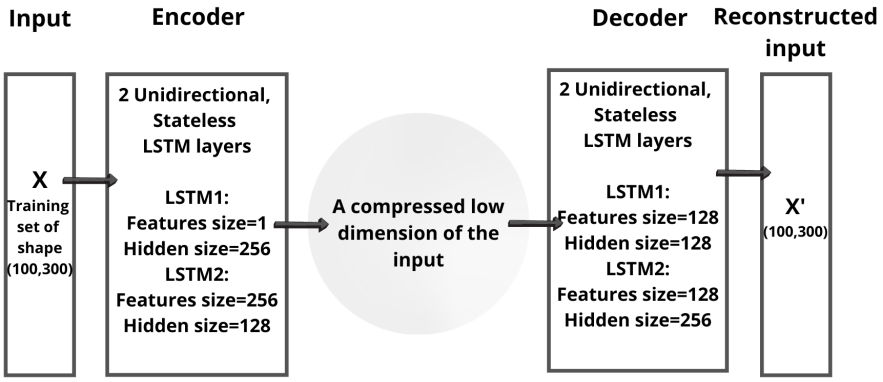


Figure 4.12: LSTM autoencoder architecture

An encoder LSTM model reads the input sequence step by step. After reading in the entire input sequence, the hidden state or output of this model represents an internal learned representation of the entire input sequence as a low fixed-length vector. This vector is then provided as an input to the decoder model that interprets it at each step and the output sequence is generated. Despite the theoretical capability of basic RNNs to model any time-varying system, in practice, their effectiveness is hampered by the difficulty of training their parameters [20]. Using LSTM models is one way for avoiding vanishing gradient problems, however, LSTM flexibility comes at the cost of higher computational and architectural complexity comparing to RC simple architecture and low complexity.

The LSTM topology gets the same input time series ECG data (limited amount of 100 windows), pass them through the model to obtain a compressed approximation of the input. The reconstruction error, for normal inputs, ideally, shall be smaller as much as possible. To classify a sequence as normal or an anomaly, we've selected a threshold from the training reconstruction error, above which a heartbeat is considered abnormal. The training comes to an end using the early stopping technique, if the reconstruction loss didn't change for n consecutive epochs, the training stops. Figure 4.13 and Figure 4.14 represent the distribution of the loss on the test normal and anomalous samples respectively and the estimated corresponding Gaussian distribution.

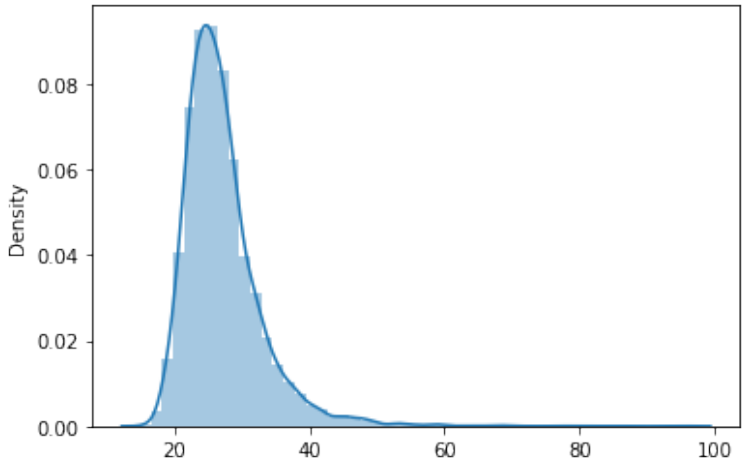


Figure 4.13: Loss distribution on the test normal samples

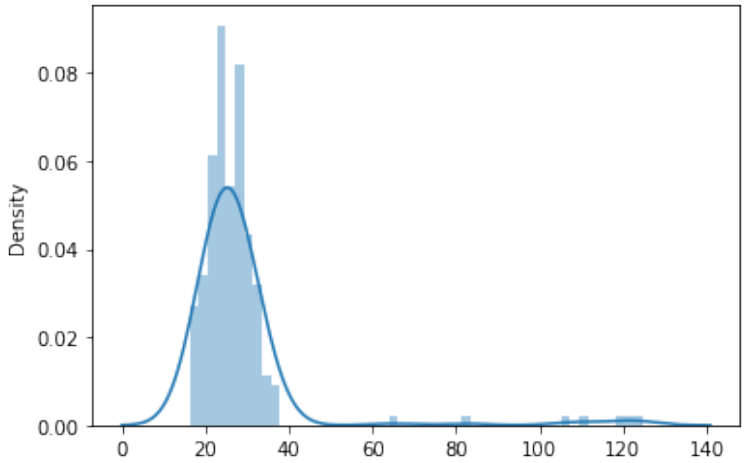


Figure 4.14: Loss distribution on the test anomalous samples

4.3 Implementation Details

In this section, we detail the specifications of the machines as well as the different software and technologies used to implement both architectures.

4.3.1 Hardware Environment

All necessary scripts related to this work have been deployed on google colab and on a machine with the following characteristics mentioned in Table 4.1.:

	Google Colab	Local PC
Operating system	Linux	Windows 10 (64-bit)
CPU	1 x single core processor @2.3Ghz	AMD Ryzen 7 4800H
RAM	12 GB	16 GB

Table 4.1: Hardware Characteristics

To run the scripts on local PC environment:
1- Create an STM environment, activate it, on anaconda and install all requirements as shown in Figure 4.15.

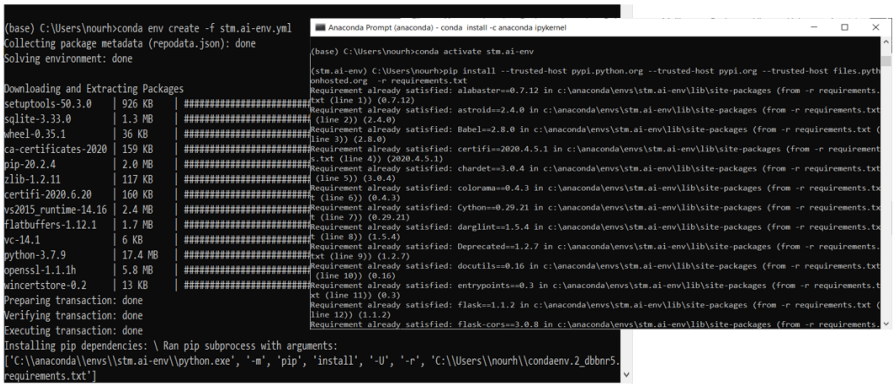


Figure 4.15: Create the environment on anaconda

2- Set this conda environment on my jupyter notebook, to do so I need to install ipykernel, Figure 4.16.

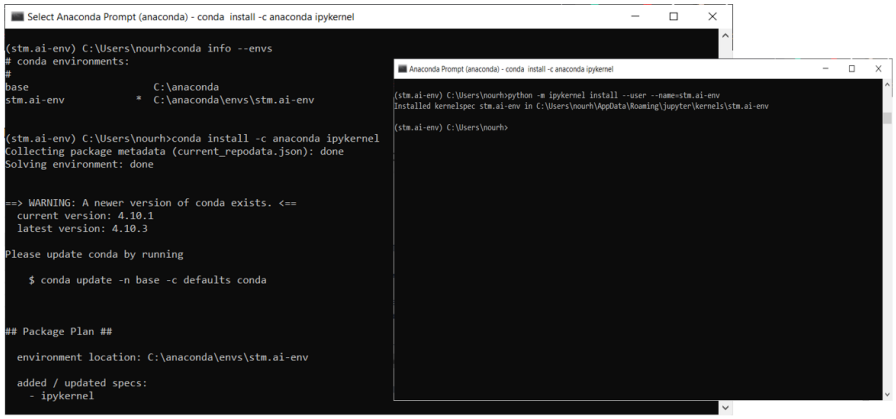


Figure 4.16: Install requirements

3- I have this conda environment in my Jupyter notebook, and I can check it on jupyter and switch between kernels Figure 4.17.



Figure 4.17: STM Kernel on Jupyter

4.3.2 Software Environment

To implement the models that we have conceived, we used a wide range of technologies and software that we list here:

Python

Python is the world's most popular and fastest-growing programming language. It comes with several very attractive machine learning libraries as well as a powerful set of packages for analytical needs. This encouraged us to choose python over the other programming languages. Furthermore, Python has a very large community that provides support through forums.

Numpy

Numpy is the fundamental package for scientific computing with Python. It allows us to process arrays and perform operations related to linear algebra.

Scikit-learn Package

Scikit-learn is one of the best-known libraries which provide a solid implementation of machine learning algorithms. In addition, Scikit-learn has efficient tools for data analysis. We mainly used scikit-learn to implement machine learning models.

Scipy

Scipy is a free and open-source Python library for mathematics, science, and engineering. It includes modules for statistics, optimization, integration, linear algebra, Fourier transforms, signal and image processing, and more
.

PyTorch

PyTorch is an open-source machine learning library based on the Torch library, used for applications such as computer vision and natural language processing, primarily developed by Facebook's AI Research lab. We used PyTorch for various reasons. In fact,

PyTorch supports using naive python code letting us write code in a more pythonic way. Moreover, PyTorch has a great set of simple APIs and it is easy to debug PyTorch code.

4.4 Conclusion

In this chapter, we presented the architectures and the models used to estimate perform the ECG anomaly detection. And, we have shown the software and hardware environments used for the implementation and testing of this project.

Chapter 5

Experimental Results and system complexity

Introduction

In this chapter, we are going to showcase the results attained after implementing the previously described architectures. To do so, we will first describe the error metrics used to evaluate these results, then the fine tune of the network's hyperparameters and finally a complexity analysis using X-CUBE-AI.

5.1 Error Metrics

In this section, we present the error metrics used to evaluate our models on the MIT BIH and STM datasets. The performance of a machine learning model is a number of measurements used to determine how much the model is capable of identifying relationships and patterns between the observations in a training dataset. Performances aren't only based on the training data but also on the model's ability to generalize over 'unseen' data.

5.1.1 Accuracy

One approach to assess how often a machine learning classification system successfully classifies a data point is to look at its accuracy. The number of correctly predicted data points among all the present data points is known as accuracy. The accuracy can also be defined more formally as the number of true (positives + negatives) divided by the number of true positives, true negatives, false positives, and false negatives (5.1). A data point that the algorithm properly predicted as true or false is called a true positive or true negative prediction respectively. A false positive or false negative, on the other hand, is a data point erroneously predicted by the model.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \qquad (5.1)$$

Accuracy is the most-used metric for evaluating classification models. The reason for its wide use is because it is easy to calculate, easy to interpret, and is a single number to summarize the model's capability.
Unfortunately, accuracy fails when dealing with imbalanced classification problems, where the distribution of classes data points in the training set isn't equal. In such a case, the best way to accurately identify the performance of the model is to use the F1-score metric with accuracy.

5.1.2 Precision & Recall

Precision is known as positive predictive value. It is a fraction of correctly predicted instances among all the instances (5.2). Precision can be considered as a measure of model quality. Higher precision means that an algorithm returns more correct predictions than wrong ones.

$$Precision = \frac{TP}{TP + FP} \qquad (5.2)$$

Recall is also called sensitivity is the fraction of correctly predicted instances by all relevant data points (5.3). When the recall value is high, our model returns most of the relevant results forgetting whether or not irrelevant data points are also returned. Figure 5.1 shows a simple graphical explanation of precision and recall.

$$Recall = \frac{TP}{TP + FN} \qquad (5.3)$$

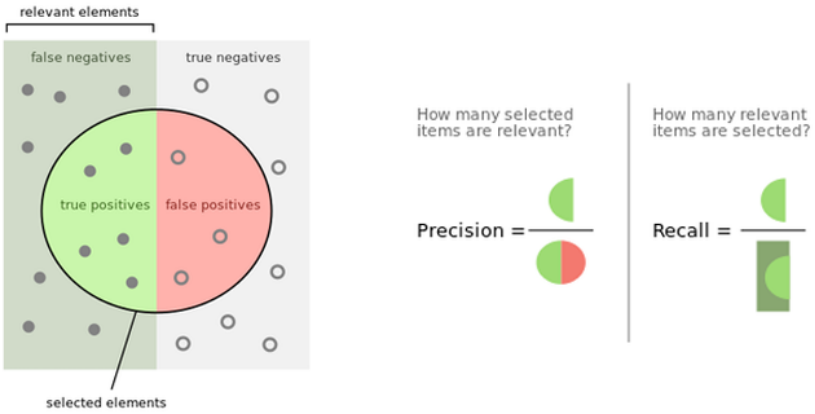


Figure 5.1: Precision and recall graphical illustration

For clarity purposes, let us put TP, FP, TN, FN in a confusion matrix for binary classification which is a 2×2 dimension matrix shown in Figure 5.2.

		Predicted	
		Negative	**Positive**
Actual	**Negative**	True Negative	False Positive
	Positive	False Negative	True Positive

Figure 5.2: Confusion matrix representation[66]

5.1.3 F1 score

F1 Score is a better approach to use when we seek a balance between Precision and Recall mostly when the data is an unbalanced distribution (Observations from one class are much higher than the observations in the other class). It is the weighted average of Precision and Recall. F1 score is calculated based on formula (5.4)

$$F1 = \frac{2 * Precision * Recall}{Precision + Recall} = \frac{2 * TP}{2 * TP + FP + FN} \qquad (5.4)$$

5.2 Experimenal results

In this section, we will understand the different hyperparameters of the HyRCoML topology, and show the process to choose the best combination of the hyperparameters.

5.2.1 Hyperparameters Vs parameters of a model

- Model parameters: The parameters that are predicted and optimized by the model from the provided training data are known as model parameters. For example, in deep neural networks, the weights of the network are called network parameters.
- Model hyperparameters: These are parameters that the model cannot estimate based on the training data. Instead, the model parameters are estimated using these hyperparameters as shown in Figure 5.3. For example, number of layers, size of the layer, the optimizer's parameters, learning rate, number of epochs...

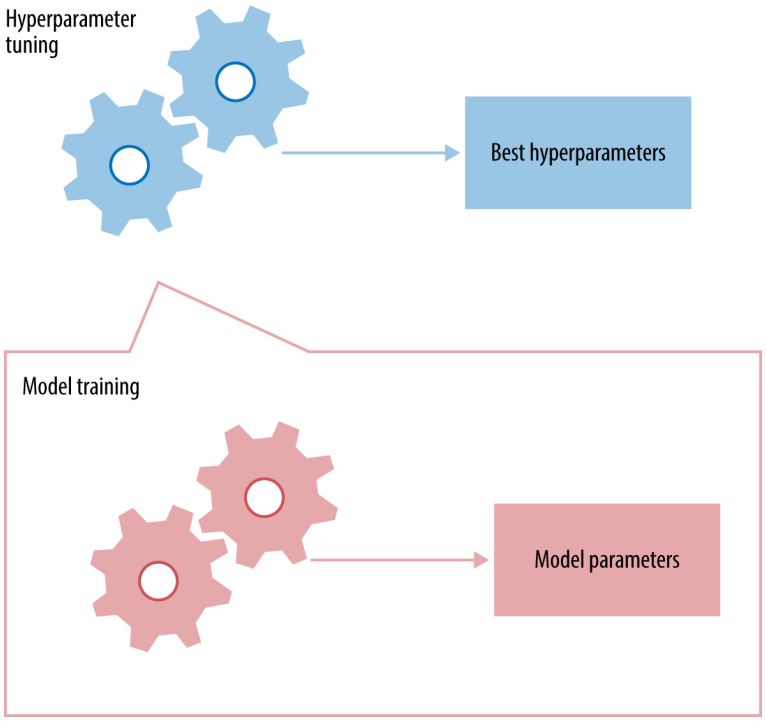


Figure 5.3: Model parameters and hyperparameters[67]

5.2.2 HyRCoML Hyperparameters

HyRCoML topology is composed by 4 modules as seen in the chapter 3. Each module has it's own hyperparameters. The choice of these hyperparameters was based on the following idea:

"Pick values that satisfies both highest performance and lowest complexity"

- The reservoir hyperparameters:

Basically, the reservoir module has 2 important hyperparameters which are the number of units inside the reservoir, the connectivity which is the percentage of nonzero recurrent connections in the bidirectional reservoir. The performance of the network was measured using a bidirectional reservoir and 25% connectivity in two different concepts when the number of internal units is equal to 100 and 150.
Results are computed for all records in the MIT BIH arrhythmia dataset. The results for record 100 and record 115 are computed for a reservoir of size 100 and 150 and shown in Table 5.1, Table 5.2, Table 5.3, Table 5.4.

Reservoir size				100			
Stride	Starting point	Shape of training data	Shape of testing data	Training time (ms)	Testing time (ms)	Accuracy	F1 score
50	105	(100, 300, 1)	(13408, 300, 1)	654.003	65621.552	0.926	0.961
50	68	(100, 300, 1)	(13408, 300, 1)	526.095	60068.489	0.894	0.944
50	219	(100, 300, 1)	(13408, 300, 1)	512.320	57357.804	0.891	0.9427
50	272	(100, 300, 1)	(13408, 300, 1)	517.114	57347.373	0.914	0.955
300	0	(100, 300, 1)	(2153, 300, 1)	554.147	9916.901	0.949	0.974

Table 5.1: Results for record 100 when reservoir size is 100

Reservoir size				150			
Stride	Starting point	Shape of training data	Shape of testing data	Training time (ms)	Testing time (ms)	Accuracy	F1 score
50	105	(100, 300, 1)	(13408, 300, 1)	940.317	98146.531	0.932	0.964
50	68	(100, 300, 1)	(13408, 300, 1)	749.118	88525.864	0.923	0.960
50	219	(100, 300, 1)	(13408, 300, 1)	750.855	89350.915	0.929	0.963
50	272	(100, 300, 1)	(13408, 300, 1)	757.784	86367.102	0.888	0.940
300	0	(100, 300, 1)	(2153, 300, 1)	724.512	15396.691	0.958	0.978

Table 5.2: Results for record 100 when reservoir size is 150

Reservoir size				100			
Stride	Starting point	Shape of training data	Shape of testing data	Training time (ms)	Testing time (ms)	Accuracy	F1 score
50	105	(100, 300, 1)	(11512, 300, 1)	552.224	63645.909	0.774	0.872
50	68	(100, 300, 1)	(11512, 300, 1)	570.514	58802.23	0.798	0.887
50	219	(100, 300, 1)	(11512, 300, 1)	533.208	62443.558	0.795	0.886
50	272	(100, 300, 1)	(11512, 300, 1)	555.405	68500.575	0.799	0.888
300	0	(100, 300, 1)	(1837, 300, 1)	622.743	9612.783	0.956	0.977

Table 5.3: Results for record 115 when reservoir size is 100

Reservoir size				100			
Stride	Starting point	Shape of training data	Shape of testing data	Training time (ms)	Testing time (ms)	Accuracy	F1 score
50	105	(100, 300, 1)	(11512, 300, 1)	768.185	92585.518	0.802	0.890
50	68	(100, 300, 1)	(11512, 300, 1)	839.478	101541.799	0.804	0.891
50	219	(100, 300, 1)	(11512, 300, 1)	743.940	96686.644	0.774	0.872
50	272	(100, 300, 1)	(11512, 300, 1)	746.076	95467.781	0.768	0.868
300	0	(100, 300, 1)	(1837, 300, 1)	824.987	14609.85	0.955	0.977

Table 5.4: Results for record 115 when reservoir size is 150

We used the following settings:
- Bidirectionality = True
- Number of reservoir units = 150
- Connectivity = 25%
- Activation function = tanh

- The PCA hyperparameters:

The PCA module will reduce the dimensionality of the input representation from 300 to a specific lower dimension. The number of principal components to compute by PCA was set to 75. The graphical visualization of the individual variance of each principal component and the cumulative variance is shown in Figure 5.4.

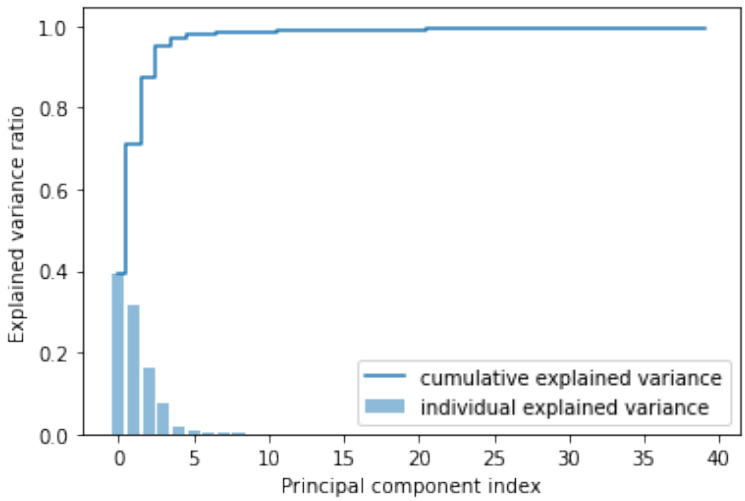


Figure 5.4: Variance and cumulative variance over the 75 principle components

The individual explained variance vector is shown in Table 5.5. It shows the carried variance by each principal axes of the data, and it describes how important that axis is in describing the data distribution. The projection of each data point onto the principal axes is called principal components of the data.

3.86764705e-01	2.64503419e-01	1.82176083e-01	1.17113419e-01	1.76558252e-02
1.34465834e-02	2.83485861e-03	1.57851109e-03	1.16684474e-03	1.02530641e-03
9.54212272e-04	6.06457121e-04	5.72250341e-04	5.10387355e-04	4.70826722e-04
3.88949527e-04	3.65027401e-04	3.39727616e-04	3.12374032e-04	2.85052694e-04
2.58731714e-04	2.53585371e-04	2.37466840e-04	2.15775362e-04	2.05716147e-04
2.00853581e-04	1.95410830e-04	1.81621697e-04	1.78825620e-04	1.71712018e-04
1.69033519e-04	1.63807621e-04	1.52224151e-04	1.46868842e-04	1.43672514e-04
1.36242321e-04	1.34699134e-04	1.31733672e-04	1.23745966e-04	1.22155659e-04
1.20201054e-04	1.16226707e-04	1.13928509e-04	1.10482055e-04	1.08079897e-04

Table 5.5: Variance for 45 principle components

0.3867647	0.6512681	0.83344424	0.95055765	0.9682135
0.98166007	0.9844949	0.98607343	0.98724025	0.9882656
0.9892198	0.98982626,	0.9903985	0.9909089	0.99137974
0.99176866	0.9921337	0.9924734	0.9927858	0.99307084
0.9933296	0.99358314	0.9938206	0.9940364	0.9942421
0.99444294	0.9946383	0.99481994	0.99499875	0.9951705
0.9953395	0.9955033	0.99565554	0.9958024	0.99594605
0.9960823	0.996217	0.99634874	0.9964725	0.9965946
0.99671483	0.99683106	0.99694496	0.9970555	0.99716353

Table 5.6: Cumulative variance over 45 principle components

We can conclude from Table 5.6 that the first 10 principal components we can reach 98% of variance, which means they account for or 98% of the overall variability within the original data. Thus, PCA's main purpose is to reduce the dimension of the given data by combining some of the original explanatory variables.
The percentage of variance explained by the PCA representation reflects the percentage of information that this representation brings about the original structure. Higher is the percentage of variance, higher is the percentage of information and less is the information loss. The hyperparameter n_components of the PCA algorithm is chosen so that the percentage of variance is big enough. The value chosen was 75 which corresponds to a percentage of variance equal to 99.9%.

- OCSVM Readout hyperparameters:

When a linear decision boundary cannot separate classes, One-Class SVM has the ability to project space through a non-linear function, lifting the data to a space with a higher dimension where a linear decision boundary does separate classes. As a result, we should adjust the kernel hyperparameter to obtain the most relevant decision boundary.
We can tune the kernel parameter by visualizing the decision boundary for each possible kernel type. However, the dimensionality of the dataset should be decreased by PCA to 2D so we can visualize the obtained decision boundary on a plot. As OCSVM is only trained on normal samples (not outliers), the produced decision boundary should cover as

many data points as possible. A new data point outside of the decision boundary would therefore be classified as an outlier. The kernel parameter has three possible values:

- ”linear”: using this parameter we can get a linear separable hyperplane (boundary). Figure 5.5 shows the decision boundary obtained for a linear kernel.

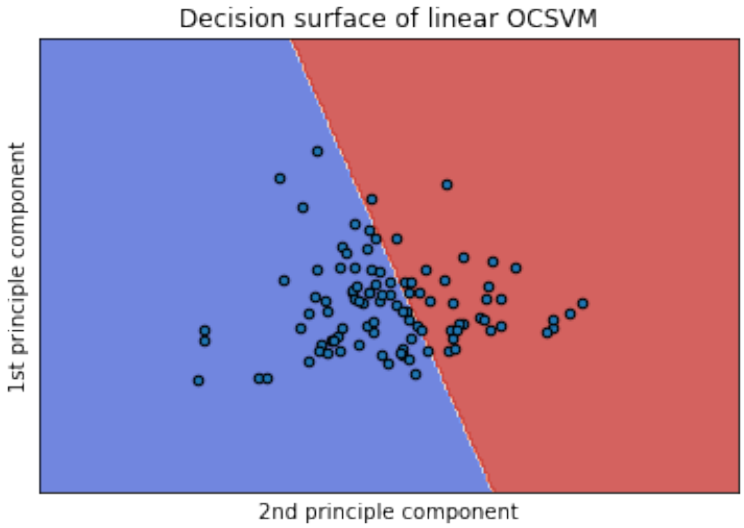


Figure 5.5: Decision boundary for One-Class SVM with a linear kernel

- ”poly”: OC-SVM generates new features by applying the polynomial combination of all the existing features and as a result, it generates a polynomial decision boundary. Figures 5.6 and 5.7 show the decision boundary obtained for a polynomial kernel with degrees 3 and 5 respectively.

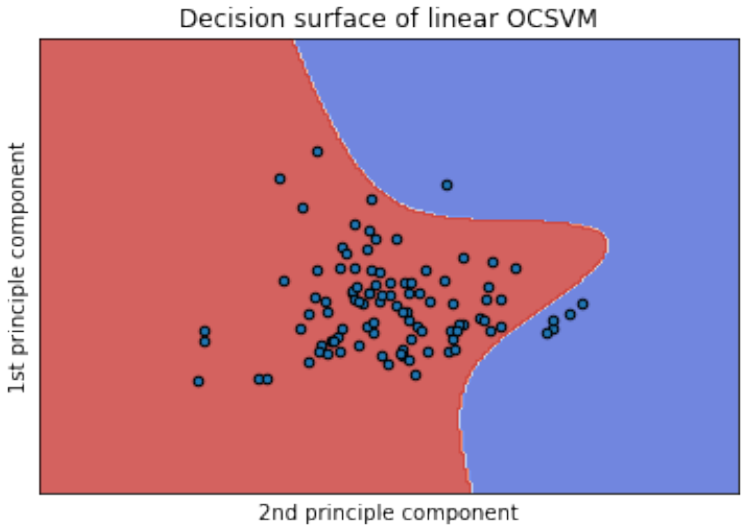


Figure 5.6: Decision boundary for One-Class SVM with a 3 degrees polynomial kernel

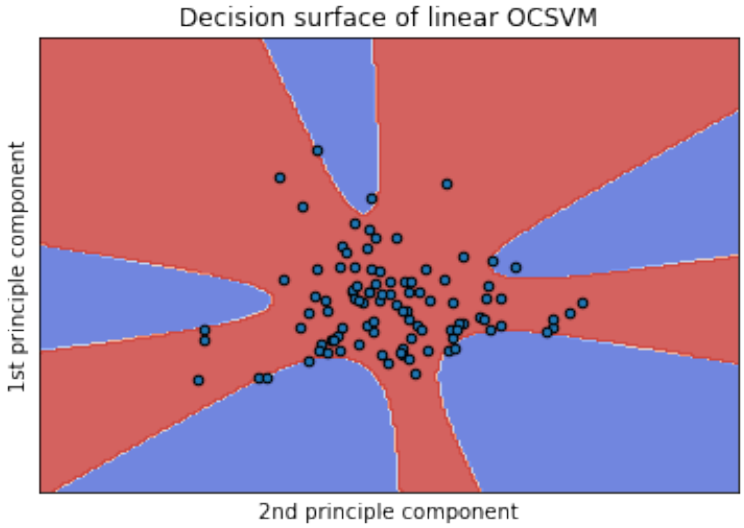


Figure 5.7: Decision boundary for One-Class SVM with a 5 degrees polynomial kernel

- ”rbf”: OC-SVM uses gaussian curves around the data points, and sums these so that the decision boundary can be defined by a type of topology condition such as curves. Figure 5.8 shows the decision boundary obtained for a radial basis function kernel.

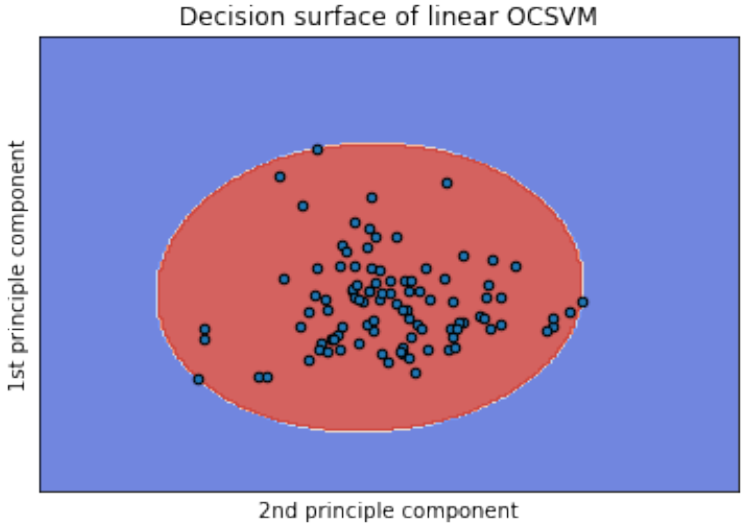


Figure 5.8: Decision boundary for One-Class SVM with a 5 degrees polynomial kernel

Because the decision boundary generated for OC-SVM with the RBF kernel includes almost all of the data points in the training dataset, and it is not overfitting the data so it is the ideal kernel to use.

5.2.3 Achieved results and interpretation

Running the experiment using the discussed settings on all the records separately are reported in the following tables:

	Baseline MIT-BIH: Starting point=0 Stride=300 Window size=300		Derived MIT-BIH: Starting point= (105,68,219,272) Stride=50 Window size=300	
Reservoir size	100	150	100	150
Mean accuracy	0.931	0.959	0.861	0.903
Mean f1 score	0.959	0.948	0.921	0.913
Variance accuracy	0.049	0.0368	0.02	0.043
Variance f1 score	0.034	0.022	0.02	0.039
Mean training time (ms)	1669.443	2114.456	1729.548	2173.178
Mean testing time (ms)	17035.31	23242.01	92504.66	131355.6

Table 5.7: Measured performances with HyRCoML network

	Baseline MIT-BIH: Starting point=0 Stride=300 Window size=300	Derived MIT-BIH: Starting point= (105,68,219,272) Stride=50 Window size=300
Mean accuracy	0.781	0.648
Mean f1 score	0.814	0.710
Variance accuracy	0.016	0.015
Variance f1 score	0.017	0.009
Mean training time (ms)	210,002.45	780,285.313
Mean testing time (ms)	25,663.463	148,077.729

Table 5.8: Measured performances with LSTM network

Tables 5.7 and 5.8 reports the mean accuracy and F1 score by training HyCoML and LSTM models respectively over all the records on the original segmented MIT BIH dataset as well as on the derived dataset obtained after sliding a window of size 300 and overlapping of 250 samples. The starting point mentioned in the table describes the starting point of the sliding window, and it should be different from 0 because we cannot guarantee the exact beginning of the heartbeat in a non-segmented real application. By this methodology, we are approaching real-world conditions.

In this experiment, we compare the anomaly detection performances obtained on the representations yielded by the RC models, and the fully trainable deep RNNs implementing LSTM cells. Both architectures are trained on a limited and fixed amount of normal data. HyRCoML resulted faster than LSTM autoencoder, for both training and test respectively of about 359 and 1.6 times. The classifiers based on fully trainable LSTM require on average more than 20 minutes on Colab with 1 single core at 2.3GHz while it only takes few seconds for the reservoir-based architecture. We can conclude that the RC model combined with these machine learning modules achieves better performances

than a fully trainable LSTM deep network in terms of accuracy, F1, training, and testing time.

The following Table 5.9 reports the results of training the HyRCoML model on the derived ST dataset with different window sizes of 100, 200, and 300. Following the achieved results, we can notice that the wiser choice of the window size to go with is 100 (considering the compromise between complexity and performance).

	Training shape	Accuracy	F1 score	Training time (ms)	Testing time (ms)
Lead 1	(100,100,1)	0.704	0.791	297.817	52211
	(200,100,1)	0.711	0.796	447.022	42534
	(300,100,1)	0.712	0.797	800.885	42839
Lead 2	(100,100,1)	0.703	0.791	243.807	42571
	(200,100,1)	0.710	0.796	445.966	41543
	(300,100,1)	0.711	0.796	776.083	42753
Lead 3	(100,100,1)	0.708	0.795	246.280	41876
	(200,100,1)	0.710	0.795	484.476	42105
	(300,100,1)	0.711	0.796	729.323	42488
Mean	(300,100,1)	0.7113	0.7963	768.936	42693

Table 5.9: HyRCoML on derived ST dataset

We can also notice that the results obtained while training and testing using the MIT BIH dataset are much better than the results obtained with ST dataset. The main difference between these two datasets is the sampling rate, which is a very important factor while working with analog signals. The MIT BIH dataset is sampled with 360 Hz, and ST dataset is sampled with 128Hz. We can conclude that a lot of information can be lost while using a smaller sampling rate, which explains the huge difference in the final results.

5.2.4 Estimated complexity

This section is dedicated to estimating the complexity on STM 32 of the HyRCoML network on inference and training process, based on the mathematics behind each module. The complexity is estimated by calculating the number of multiplications and additions (MACC) in each module.

5.2.4.1 Estimated complexity of the training process

- Reservoir module complexity

Input shape = (100,300,1)
Output shape = (100,300,300) Reservoir size= 150
W_{in} is of shape=(150,1) W_r is of shape=(150,150) Previous_state is of shape =(150,1)

X_t is one value at any time step
The output for this network is: $W_{in} * X + W_r * sparsity * previous_state$
$Complexity = 150 * 1 + 150 * 0.25 * 150 = 5775 \quad MACC$
These are the MACC for processing inputs in one time direction. But the reservoir is bidirectional, so for the reverse pass there are other 5775 MACC.

In total: $Complexity\, of\, bidirectional\, reservoir = 2 * 5775 = 11,550 \quad MACC$
$\forall \quad 1 <= X_t <= 300$

- Representation module

Input shape = (100,300,300)
Output shape = (100,300)
This module is with 0 complexity as it only picks the last state or the mean of all the reservoir states.

- PCA module

Input shape = (100,300) = (N,P)
Output shape = (100,75)
The complexity of computing the Covariance matrix is $= N * P^2 = 9,000,000 \quad MACC$
The complexity of the eigen-value decomposition $= P^3 = 27,000,000 \quad MACC$
The complexity of computing new vectors $= 300 * 100 * 75 = 2,250,000 \quad MACC$

In total: $Complexity\, for\, all\, training\, samples = 38,250,000 \quad MACC$

- OCSVM module

Input shape = (100,75)
OCSVM uses RBF kernel to project the data into a higher space dimension (100,100)
The kernel will compute the euclidean distance between the 100 observations:
$\forall\ i < j <= 100$ we compute the euclidean distance $(X[i], X[j])$
Complexity of computing the kernel $= (100 * 100 - 100)/2 * (75\, MACC + 75\, additions) = 371,250\, MACC + 371,250\, additions$ as $\quad dist(x,x) = 0 \quad and \quad dist(x,y) = dist(y,x)$
Complexity for training the SVM = 100*100*100= 1,000,000 MACC

In total : $Complexity = 1,371,250 \quad MACC + 371,250 \quad additions$

Module	Training complexity
Reservoir module	11,550 MACC 11,550*(300*100 windows)=345,000,000 MACC
Representation module	0 MACC
PCA module	38,250,000 MACC
Readout module: OC-SVM	1,371,250 MACC+ 371,250 additions
Total	**382,371,250 MACC+371,250 additions**

Table 5.10: Estimated complexity of training

The estimated complexity of the training process for all modules of the HyRCoML network is reported in Table 5.10.

5.2.4.2 Estimated complexity in inference mode

- Reservoir module

Input shape = (1,300)
Output shape = (300,300)
$Complexity/s = 2 * 300 * (150 * 1 + 150 * 150 * 0.25) * 360/300 = 4,158,000 \quad MACC/s$

- PCA module

Input shape = (1,300) Output shape = (1,75) $Complexity/s = (300 * 75) * 360/300 = 27,000 \quad MACC$

- OCSVM module

Input shape = (1,75) $Complexity/s = (75\,MACC{+}75\,additions){*}360/300 = 90\,MACC/s{+}90, additions/s$

Module	Inference complexity
Reservoir module	4,158,000 MACC/s
Representation module	0 MACC/s
PCA module	27,000 MACC
Readout module: OC-SVM	90 MACC/s+90 additions/s
Total	**4,185,090 MACC/s +90 additions/s**

Table 5.11: Estimated complexity on inference mode

The estimated complexity on inference mode for all modules of the HyRCoML network is reported in Table 5.11.

5.2.4.3 Estimated complexity and memory on STM32

Table 5.12 below reports the estimated memory and time complexity of the proposed HyR-CoML model on both STM 32 Cortex M4 and STM32 Cortex M7. The main difference between STM32 Cortex M4 and STM32 Cortex M7 is the maximum CPU frequency. ARM 32-bit Cortex M4 core can run up to 120 MHz. While ARM 32-bit Cortex M7 core has been designed to deliver a very high level of performance while maintaining the excellent responsiveness and ease-of-use that can run up to 480MHz.

Input memory for training task (bytes)	10.000= 300*2*100 / 6 (overlap ratio by 50 samples in MIT BIH)	
Input memory for inference task (bytes)	600 = 300 samples * 2 bytes	
	STM32 ARM Cortex M4 (80MHz)	**STM32 ARM Cortex M7 (480MHz)**
Mean training time (s)	43.016	4.779
Mean testing time (s)	0.471	0.052

Table 5.12: Estimated testing, training time and memory on STM32

5.2.5 Measured complexity by X-CUBE-AI

5.2.5.1 X-CUBE-AI definition

X-CUBE-AI is an STM32Cube Expansion Package part of the STM32Cube [68]. AI ecosystem and extending STM32CubeMX capabilities, generate optimized library within the user-s project, with automating the conversion of pre-trained AI algorithms, including both Neural Network and Machine Learning models. The X-CUBE-AI Expansion Package offers also several means to validate Artificial Intelligence algorithms both on desktop PC and STM32, as well as to measure performance on STM32 devices without user handmade ad hoc C code. Furthermore, X-CUBE-AI provides a scalable infrastructure that automates much of the testing and training process for classical machine learning algorithms.

5.2.5.2 X-CUBE-AI features

The main features of the X-CUBE-AI package are:

- Generation of an STM32-optimized library from pre-trained Neural Network and classical Machine Learning models.
- Native support for various Deep Learning frameworks such as Keras and TensorFlow Lite, and support for all frameworks that can be exported to the ONNX standard format such as PyTorch and more.

- Support for various built-in scikit-learn models such as support vector machine (SVM), , K-means, Random forest and more.
- Allows the use of larger networks by storing weights in external Flash memory and activation buffers in external RAM.
- Free, user-friendly license terms.

5.2.5.3 ONNX

ONNX is an open-source tool in AI used as an intermediary machine learning framework in order to convert between different machine learning frameworks. So let's provide an example, if we want to convert the model from TensorFlow to TensorRT, or from Pytorch to TFLite, or any other machine learning frameworks presented in Figure 5.9. ONNX is considered a good intermediary to use to convert the model. So ONNX is very well designed to basically implement all different kinds of neural network functions and functionalities in these ML models, so it can support this cross-functionality to have a baseline, common framework to convert into.



Figure 5.9: ONNX tool for AI models conversion[20]

5.2.5.4 HyRCoML complexity analyses

In order to analyze the complexity of the proposed HyRCoML topology and to verify its estimated complexity, we used the X-CUBE-AI library. As figure 5.10 shows, the model is considered as a composition of two models which are the reservoir and a pipeline composed of PCA and OCSVM models. From the beginning, the reservoir was implemented using NumPy and SciPy, the fundamental packages for mathematical computing. It was designed as a matrix multiplication considering the sparsity of the matrix and all needed initialization. However, to convert the reservoir implementation into an ONNX file, an intermediate conversion to Keras is mandatory as ONNX doesn't support customized models created by the user. The recurrent reservoir network can be considered as a single nontrainable recurrent layer initialized in a special way to ensure sparsity and input scaling. Having the reservoir as a Keras model, it becomes easy to convert the reservoir into an ONNX file using tf2onnx python package.

The second part of the network is composed of PCA and OCSVM, two models that were implemented using Sickit-learn were converted to ONNX format using skl2onnx python package.

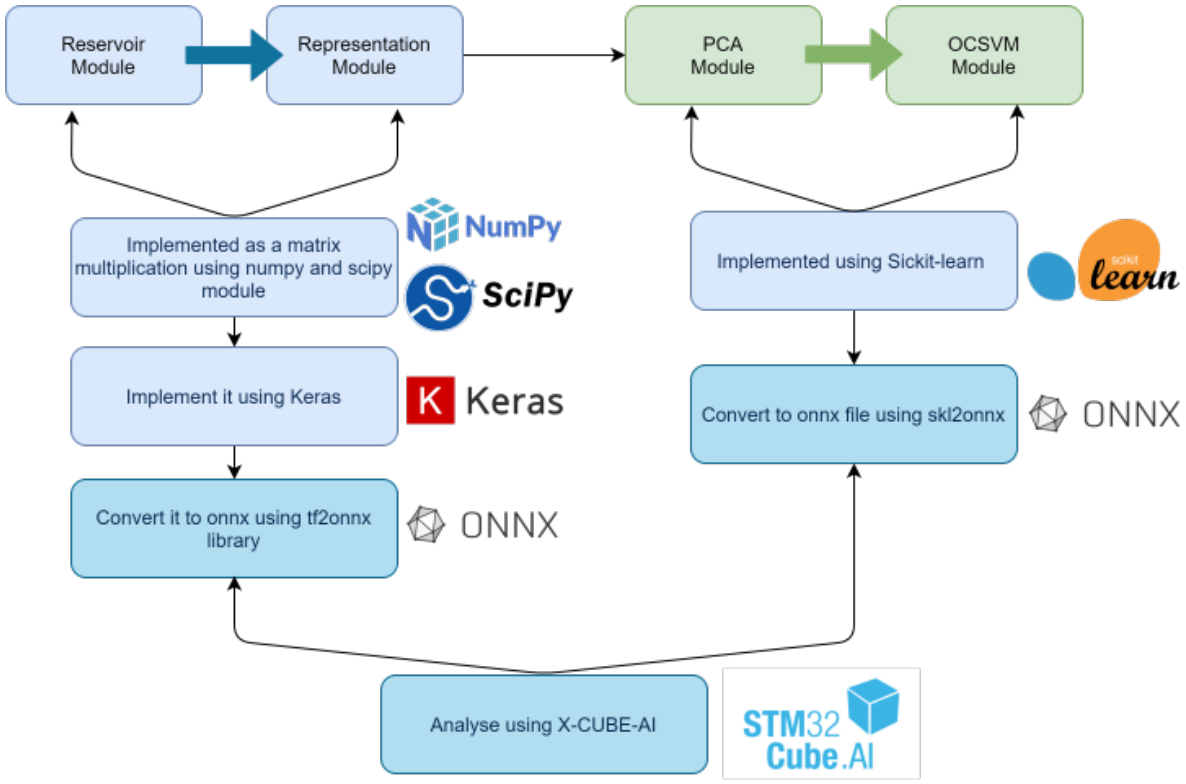


Figure 5.10: HyRCoML and complexity analysis diagram

After a successful conversion to ONNX format, both files were treated separately and dependently (Output of the reservoir is the input of the pipeline) on X-CUBE-AI to analyze and measure the complexity of the HyRCoML on STM32 microcontroller. The currently obtained complexity is shown in Figure 5.11 and Figure 5.12 for both the reservoir and the PCA+OCSVM pipeline respectively. We can conclude that the measured complexity is pretty much similar to the estimated complexity.

```
Neural Network Tools for STM32AI v1.5.1 (STM.ai v7.0.0-RC8)
Created date          : 2021-08-30 11:57:56
Parameters            : analyze -m PCA+OCSVM.onnx

Exec/report summary (analyze)
------------------------------------------------------------------------------------------------------------------------
model file            : C:\Users\Norhen\Downloads\PCA+OCSVM.onnx
type                  : onnx
c_name                : network
workspace dir         : C:\Users\Norhen\Downloads\stm32ai_ws
output dir            : C:\Users\Norhen\Downloads\stm32ai_output

model_name            : PCAOCSVM
model_hash            : 2c9cb7b7dda57fa0ca07f40902a11fda
input                 : float_input [300 items, 1.17 KiB, ai_float, float, (1, 1, 1, 300)]
inputs (total)        : 1.17 KiB
output                : label [1 items, 4 B, ai_i32, integer, (1, 1, 1, 1)]
output                : SVMO1 [1 items, 4 B, ai_float, float, (1, 1, 1, 1)]
outputs (total)       : 8 B
params #              : 24,168 items (99.75 KiB)
macc                  : 25,842
weights (ro)          : 96,672 B (94.41 KiB) / -5,472(-5.4%) vs original model
activations (rw)      : 1,500 B (1.46 KiB)
ram (total)           : 2,708 B (2.64 KiB) = 1,500 + 1,200 + 8

Model name - PCAOCSVM ['float_input'] ['label', 'scores']
------------------------------------------------------------------------------------------------------------------------
id   layer (type)                   oshape    param/size      macc     connected to      |   c_size           c_macc        c_type
------------------------------------------------------------------------------------------------------------------------
0    mean (Placeholder)             (c:300)   300/1,200                                  |   -1,200(-100.0%)
     float_input (Input)            (c:300)                                              |
     sub_result (Eltwise)           (c:300)                   300      float_input       |   +1,200(+100.0%)              eltwise()[0]
                                                                       mean              |
------------------------------------------------------------------------------------------------------------------------
1    variable (Dense)               (c:75)    22,500/90,000   22,575   sub_result        |                                dense()[1]
------------------------------------------------------------------------------------------------------------------------
2    SVMO1 (SVMRegressor)           (c:1)     1,368/10,944    2,964    variable          |   -5,472(-50.0%)               svmreg()/o[2]
------------------------------------------------------------------------------------------------------------------------
3    scores (Cast)                  (c:1)                     2        SVMO1             |                    -2(-100.0%)
------------------------------------------------------------------------------------------------------------------------
4    float_prediction (Nonlinearity) (c:1)                    1        scores            |                                nl()[3]
------------------------------------------------------------------------------------------------------------------------
```

Figure 5.11: X-CUBE-AI results for PCA+OCSVM

Figure 5.12: X-CUBE-AI results for PCA+OCSVM

5.3 Conclusion

During this chapter, we have successfully implemented a fixed complexity tiny reservoir heterogeneous network that takes an input ECG signal and predicts its normality and abnormality. A fully trainable LSTM network was also implemented to perform the same task using the same amount of data. Results for both architectures were provided. We have concluded that the reservoir-based approach outperforms the benchmark LSTM network in terms of performance and complexity. And finally, we completed the project by analyzing the complexity on STM32 microcontrollers using specific tools.

General Conclusion

General Conclusion

This report outlines the work accomplished as part of a graduation internship project. It's titled Fixed complexity tiny reservoir heterogeneous network for online ECG learning of anomalies and was developed at STMicroelectronics Milan, Italy during a six-month period.

During the project, we proposed a bidirectional RC with one class classifier based on the recurrent reservoir model space representation, which can use a limited and fixed amount of normal data to perform real-time anomaly detection on time series ECG data generated by a sliding window. We concluded that recurrent reservoir networks combined with these machine learning modules achieve better performances both in terms of classification accuracy and execution time, that is competitive to those obtained by more complex models, while requiring only a few seconds to be trained online. This makes this work suitable for fast personalization and real-time embedded applications.

We start by presenting our project description, which includes the hosting organism, the problem statement, and the proposed solution's main ideas. Then we focus on specifying the state-of-the-art AI-based techniques and the theoretical part needed for a better understanding of the project. After that, we present in detail the data and its preparation approaches used to train and evaluate the work. Then, it comes the star of the show, the HyRCoML designed architecture that was explained in depth. Finally, we display our achieved work as well as the decisions that we went into during the modeling, evaluation, and model complexity phases.

It is fair enough to state that we were able to overcome all of the difficulties that occurred during the work process. But, even that our project was accomplished successfully and achieved good results, there's still some work to do to enhance and to improve the proposed solution by performing different tests using different re-labeling criteria for the dataset, and also by focusing on multiple reservoir hyperparameters.

During this internship, I've had a great opportunity of working and familiarizing myself with state-of-the-art technologies, in addition to discovering the fields of online machine learning and deep learning models. I had the chance, with the support of both my ST supervisors and my university supervisors, to put the knowledge base I gained while studying at Sup'Com and UNIPD in a flourishing and real-world work environment.

Accepted Paper

[1] N. Abdennadher, D. Pau, A. Bruno, "Fixed Complexity Tiny Reservoir Heterogeneous Network for On-Line ECG Learning of Anomalies" accepted in 2021 IEEE 10th Global Conference on Consumer Electronics (GCCE).

A presentation of the proposed work will take place in Kyoto 12-15 October 2021.

Fixed complexity tiny reservoir heterogeneous network for on-line ECG learning of anomalies

Norhen Abdennadher
Universita di Padova, Italy
SUPCOM, Tunisia
STMicroelectronics, Italy
norhen.abdennadher@studenti.unipd.it

Danilo Pau, FIEEE
STMicroelectronics
Agrate Brianza, Italy
danilo.pau@st.com

Arcangelo Bruna
STMicroelectronics
Catania, Italy
arcangelo.bruna@st.com

***Abstract*— The electrocardiogram (ECG), being one of the most extensively used signals to monitor cardiovascular diseases (CVDs), captures the heart's arrhythmias. Patients with such pathology are often monitored for extended periods of time, requiring data storage, and a very time-consuming off-line search of anomalies. This is especially inefficient when indicative patterns in the biological signals are infrequent, requiring more analysis time of medical doctors, and entailing a difficult visual search task for the diagnosis.**

In this paper, we propose an automated deep learning pipeline based on reservoir computing (RC), followed by principal component analysis (PCA) and one-class support vector machine (OC-SVM), that can be used to perform on-line learning and real-time anomaly detection of pathological conditions and therefore to raise warnings. The learning step requires fixed computational complexity and memory that are affordable on a low-power microcontroller unit (MCU). During the learning phase, it uses a very limited amount of input data, e.g., 10 KB, which makes this work suitable for fast personalization when device restart is required. The detection accuracy has been evaluated on the publicly available MIT-BIH arrhythmia dataset, also modified to mimic temporal sliding of input tensors on the ECG streaming data. Best F1 score and accuracy are 91.5%, 95.4% with variance over the processed data of 0.02.

Keywords*— *ECG, time series data, reservoir computing, recurrent network, anomaly detection, principal component analysis, one class support vector machine, sliding window.

To automatically detect heart arrhythmias in an ECG stream, an algorithm must implicitly recognize the distinct wave types and discern the complex relationships between them over time. This is difficult to achieve by hand-crafted rules due to the variability in wave morphology among patients. Hence, a real-time online learning approach that uses a few bunches of data and achieves adequate performances would be ideal. Moreover, to make such an approach broadly available, it should be implementable on a low-cost MCU. Usually, it features limited computational power (in the order of tens of MHz) and limited memory (hundreds of Kbytes) in 10 to 100s µW/MHz power consumption envelope.

In this paper, we focus on reservoir recurrent neural networks (RRNNs), which first processes sequentially the time series (TS) ECG data with a dynamically learnt model using limited amount of input data. RC is a family of NNs which recurrent weights are set randomly, and they are kept fixed at inference time [7]. Despite this drastic simplification, which doesn't require neither back propagation, nor stochastic gradient descent learning, as well as multiple epoch runs, RC provides a rich pool of dynamic features that are suitable for solving a large variety of tasks. Leveraging it, we propose an unsupervised procedure for RRNN online training by using limited in size, anomaly-free time-series data, using the four modules pipelined architecture: RC, representation module, dimensionality reduction, followed by a readout layer.

In this study, we compare another NN architecture

Bibliography

[1] Manivannan Rangasamy. "Role of telemedicine in health care system: a review." In: *Int J Recent Adv Pharma Res* 2 (2011), pp. 1–10.

[2] W. Van Lerberghe. "The world health report 2008: primary health care: now more than ever." In: *World Health Organization,* (2008).

[3] A. Abajobir M. Naghavi C. Abbafati. "Global, regional, and national age-sex specific mortality for 264 causes of death, 1980–2016: a systematic analysis for the global burden of disease study 2016." In: *The Lancet* 390 (2017), pp. 1151–1210.

[4] A. Cohen-Solal K. Dickstein. "ESC guidelines for the diagnosis and treatment of acute and chronic heart failure 2012: The task force for the diagnosis and treatment of acute and chronic heart failure 2012 of the european society of cardiology. developed in collaboration with the heart." In: *European journal of heart failure* 14 (2012), pp. 803–869.

[5] S. C. Achuff S. H. Gottlieb. "Silent Ischemia on Holter Monitoring Predicts Mortality in High-Risk Postinfarction Patients." In: *JAMA* 259 (1988), pp. 1030–1035.

[6] *STM around the world.* URL: https://www.st.com/resource/en/company_presentation/company_presentation.pdf.

[7] *STMicroelectronics latest solutions.* URL: https://newsroom.st.com/media-center/press-item.html/t4271.html.

[8] A. A. Abajobir M. Naghavi. "Global, regional, and national age-sex specific mortality for 264 causes of death, 1980–2016: a systematic analysis for the global burden of disease study 2016." In: *The Lancet* 390 (2017), pp. 1151–1210.

[9] J. J. McMurray A. F. Members. "ESC guidelines for the diagnosis and treatment of acute and chronic heart failure 2012: The task force for the diagnosis and treatment of acute and chronic heart failure 2012 of the european society of cardiology. developed in collaboration with the heart failure association (HFA) of the ESC." In: *European journal of heart failure* 14 (2012), pp. 803–869.

[10] P. Ouyang M. L. Weisfeldt. "Silent ischemia as a marker for early unfavorable outcomes in patients with unstable angina." In: *New England Journal of Medicine* 314 (1986), pp. 1214–1219.

[11] A. Cohen-Solal K. Dickstein. "Emotional response and changes in heart rate variability following art-making with three different art materials." In: *New England Journal of Medicine* (2018), p. 968.

[12] D.-C. Cheng S.-H. Liu. "Arrhythmia identification with two-lead electrocardiograms using artificial neural networks and support vector machines for a portable ECG monitor system." In: 13 (2013), pp. 813–828.

[13] T. Ince S. Kiranyaz. "Real-time patient-specific ECG classification by 1-D convolutional neural networks." In: *IEEE Transactions on Biomedical Engineering* 63 (Mar. 2016), pp. 664–675.

[14] J. Zhou N. Wang. "Energy-efficient intelligent ECG monitoring for wearable devices." In: *IEEE Transactions on Biomedical Circuits and Systems* (2019), pp. 1–1.

[15] E. D. beyli. "ECG beats classification using multiclass support vector machines with error correcting output codes." In: *Digital Signal Processing* 17 (2007), pp. 675–684.

[16] L. M. Flores Nava O. A. Crdenas. "ECG arrhythmia classification based on fuzzy cognitive maps." In: *in 2019 16th International Conference on Electrical Engineering, Computing Science and Automatic Control (CCE)* 17 (2019), pp. 1–4.

[17] Leonard S Lilly. "Pathophysiology of Heart Disease: A Collaborative Project of Medical Students and Faculty (sixth ed.)" In: *Lippincott Williams Wilkins* 978 (2007), p. 74.

[18] H. Jaeger M. Lukoševičius. "Reservoir computing approaches to recurrent neural network training." In: *Computer Science Review* 3 (2009), pp. 127–149.

[19] *AI projects management methodology*. URL: https://appinventiv.com/blog/ai-project-management/#1.

[20] *ECG waveform segments*. URL: https://www.alivecor.com/education/ecg.html.

[21] E.J. Gialafos P.E. Dilaveris. "Simple electrocardiographic markers for the prediction of paroxysmal idiopathic atrial fibrillation." In: *American Heart Journal* 135 (1998), pp. 733–738.

[22] B. Eskofier M. Elgendi. "Revisiting QRS detection methodologies for portable, wearable, battery-operated, and wireless ECG systems PLoS ONE." In: *American Heart Journal* 9 (2014), e84018.

[23] T. Ince S. Kiranyaz. "Real-time patient-specific ecg classification by 1-d convolutional neural networks." In: *IEEE Transactions on Biomedical Engineering* 63 (2016), pp. 664–675.

[24] A. Cohen-Solal K. Dickstein. "Emotional response and changes in heart rate variability following art-making with three different art materials." In: *Frontiers in psychology* 9 (2018), p. 968.

[25] *Weather conditions*. URL: https://www.mountelizabeth.com.sg/healthplus/article/arrhythmia-guide.

[26] *Stock exchange*. URL: https://www.influxdata.com/what-is-time-series-data/.

[27] *Heart rhythm morphology*. URL: https://www.influxdata.com/what-is-time-series-data/.

[28] *Health monitoring.* URL: https://www.influxdata.com/what-is-time-series-data/.

[29] Arthur Zimek. "Outlier Detection." In: *Encyclopedia of Database Systems, Springer New York* (), pp. 1–5.

[30] *Supervised anomaly detection.* URL: https://towardsdatascience.com/a-note-about-finding-anomalies-f9cedee38f0b.

[31] *Semi Supervised anomaly detection.* URL: https://towardsdatascience.com/a-note-about-finding-anomalies-f9cedee38f0b.

[32] *Unsupervised anomaly detection.* URL: https://towardsdatascience.com/a-note-about-finding-anomalies-f9cedee38f0b.

[33] R. Pálovics L. Kocsis. "Online machine learning in big data streams: overview." In: *Springer Link* (Feb. 2019), pp. 1–5.

[34] *Biological neuron vs. artificial neuron.* URL: https://cs231n.github.io/neural-networks-1/.

[35] *Artificial neural network.* URL: https://www.ibm.com/cloud/learn/neural-networks.

[36] *Minimizing a loss function.* URL: https://www.ibm.com/cloud/learn/neural-networks.

[37] *Recurrent neural network.* URL: https://www.ibm.com/cloud/learn/neural-networks.

[38] R. Mark G. Moody. "The MIT-BIH arrhythmia database on CDROM and software for use with it." In: *Proceedings Computers in Cardiology* (1990), pp. 185–188.

[39] *ECG signals in record 205 for both channels.* URL: https://www.researchgate.net/publication/324457712_A_deep_learning_approach_for_ECG-based_heartbeat_classification_for_arrhythmia_detection.

[40] *Modified limb lead II electrode placement.* URL: https://www.researchgate.net/publication/304617942_Modified_limb_lead_ECG_system_effects_on_electrocardiographic_wave_amplitudes_and_frontal_plane_axis_in_sinus_rhythm_subjects.

[41] R. G. Mark Moody. "The impact of the MIT-BIH arrhythmia database." In: *IEEE Engineering in Medicine and Biology Magazine* 20 (2001), pp. 45–50.

[42] Fabi Badilini. "The ISHNE holter standard output file format." In: *Annals of non-invasive electrocardiology* 3 (1998), pp. 263–266.

[43] T. W J. Pan. "Real time algorithm detection for QRS." In: *IEEE Trans. Eng. Biomed Eng* 32 (1985), pp. 230–236.

[44] Sabarimalai M. Manikandan. "A novel method for detecting R-peaks in electrocardiogram (ECG) signal." In: *Biomedical Signal Processing and Control* 7 (2012), pp. 118–128.

[45] *Sliding window overlapping.* URL: http://vishnuviswanath.com/flink_streaming.

[46] Bernard J Gersh. "A novel method for detecting R-peaks in electrocardiogram (ECG) signal." In: *Mayo Clinic Heart Book* (2000), pp. 6–8.

[47] *WFDB python package*. URL: https://wfdb.readthedocs.io/en/latest.

[48] *The ElementTree XML API*. URL: https://docs.python.org/3/library/xml.etree.elementtree.html.

[49] *ISHNE Holter Library for Python*. URL: http://thew-project.org/papers/Badilini.ISHNE.Holter.Standard.pdf.

[50] *ISHNE Holter project*. URL: https://pypi.org/project/ishneholterlib/.

[51] F. M. Bianchi K. Ø. Mikalsen. "Learning similarities between irregularly sampled short multivariate time series from EHRs." In: (2016).

[52] S. Scardapane F. M. Bianchi. "Reservoir computing approaches for representation and classification of multivariate time series." In: *IEEE transactions on neural networks and learning systems* 32 (May 2020), pp. 2169–2179.

[53] E. Maiorino F. M. Bianchi. "Recurrent neural networks for short-term load forecasting: an overview and comparative analysis." In: *Mayo Clinic Heart Book* (2017).

[54] S. C. Kremer F. Kolen. "A field guide to dynamical recurrent networks." In: *John Wiley Sons* (2001).

[55] C. Gallicchio. "Sparsity in reservoir computing neural networks." In: *International Conference on INnovations in Intelligent SysTems and Applications (INISTA) IEEE* (Aug. 2020), pp. 1–7.

[56] A. F. Sheta A. Rodan. "Bidirectional reservoir networks trained using SVM + privileged information for manufacturing process modeling." In: *Soft computing* 21 (2017), pp. 6811–6824.

[57] S. Jürgen A. Graves. "Framewise phoneme classification with bidirectional LSTM and other neural network architectures." In: *Neural networks* 18 (2005), pp. 602–610.

[58] F. M. Bianchi S. Løkse. "Training echo state networks with regularization through dimensionality reduction." In: *Cognitive Computation* 9 (2017), pp. 364–378.

[59] Mexicano Rodríguez. "Feature extraction of electrocardiogram signals by applying adaptive threshold and principal component analysis." In: *Journal of applied research and technology* 13 (2015), pp. 261–269.

[60] C. J. Burges. "A tutorial on support vector machines for pattern recognition." In: *Data mining and knowledge discovery* 2 (1998), pp. 121–167.

[61] David M.J. Tax. "Uniform object generation for optimizing one-class classifiers." In: *Journal of Machine Learning Research 2* 2 (2001), pp. 155–173.

[62] *Illustration of OC-SVM hypersphere*. URL: https://medium.com/@mail.garima7/one-class-svm-oc-svm-9ade87da6b10.

[63] *Decision boundary not very 'curvy' with small Gamma*. URL: https://chrisalbon.com/code/machine_learning/support_vector_machines/svc_parameters_using_rbf_kernel/#:~:text=gamma%20and%20C%20.-,Gamma,decision%20region%20is%20very%20broad..

[64] *Decision boundary very 'curvy' with big Gamma*. URL: https://chrisalbon.com/code/machine_learning/support_vector_machines/svc_parameters_using_rbf_kernel/#:~:text=gamma%20and%20C%20.-,Gamma,decision%20region%20is%20very%20broad..

[65] *LSTM Cell and It's Operations*. URL: https://towardsdatascience.com/illustrated-guide-to-lstms-and-gru-s-a-step-by-step-explanation-44e9eb85bf21.

[66] *Confusion matrix representation*. URL: https://towardsdatascience.com/accuracy-precision-recall-or-f1-331fb37c5cb9.

[67] *Model parameters and hyperparameters*. URL: https://www.oreilly.com/library/view/evaluating-machine-learning/9781492048756/ch04.html.

[68] *X-CUBE-AI Expansion Package*. URL: https://www.st.com/en/embedded-software/x-cube-ai.html.